ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC VÀ KỸ THUẬT THÔNG TIN



GOOGLE SKILLS SCRAPER

Trích Xuất Nội Dung Của Google Skills Vào Kho Kiến Thức Cá Nhân (PKB)

Mã Môn:	IE221
Tên Môn:	Kỹ Thuật Lập Trình Python
Lớp:	IE221.F33.LT.CNTT
Giảng Viên:	ThS. Nghi Hoàng Khoa
MSSV:	25410291
Sinh Viên:	Đinh Xuân Sâm

Tháng 09/2026

Lời Nói Đầu

Google Skills (skills.google), trước đây là Google Cloud Skills Boost, là nền tảng đào tạo e-Learning chính thức của Google, bởi Google và cho các sản phẩm/dịch vụ của Google.

Vốn là nền tảng tập trung cho Google Cloud, nhưng bây giờ Google Skills đã mở rộng đối tượng sản phẩm phục vụ, và từ đó mở rộng số lượng nội dung được đăng tải, lên tới hàng trăm lộ trình học tập (path), hàng nghìn khoá học (course) và bài thực hành (lab).

Cách tổ chức của Google Skills cơ bản cũng tương tự các nền tảng e-learning khác, bao gồm: Lộ Trình Học Tập (Khóa Học) → Các Môn Học → Các Bài Học (Video, Bài Đọc, Bài Lab, vv..) và Các Bài Kiểm Tra (Quiz), Chứng Chỉ (Badge), vv.. rất bài bản và nhiều cấp bậc.

Phát sinh từ thực tế từ công việc cần làm việc trực tiếp với các dịch vụ của Google, cụ thể là Google Cloud, sinh viên thực hiện cũng cần sử dụng Google Skills để nhận các kiến thức liên quan về sản phẩm và dịch vụ, là một phần trong quá trình đào tạo (enablement) của công ty.

Tuy nhiên, sinh viên nhận thấy cách trình bày nội dung của Google Skills rất khó để theo dõi ở một số khía cạnh, ví dụ: Một bài học (course) hoặc bài lab có thể được sử dụng trong rất nhiều khóa học, tuy nhiên Google Skills không biểu diễn trực quan được mối quan hệ đó, khiến việc hình dung mối liên hệ kiến thức giữa các khóa học là rất khó, hoặc không thể, chỉ có thể tham gia hoặc đi qua từng khóa học để nhận ra là có môn học được dùng lại.

Đồng thời, việc bắt buộc sử dụng website để duyệt nội dung bài học, vốn chủ yếu là video youtube, không phải dạng văn bản, từ đó không hoặc khó cho phép việc ghi chú, tóm tắt, đánh dấu, vv.. về cơ bản là một cách tiếp cận không quá hiệu quả hay rất khó tham khảo lại.

Vì vậy, sinh viên đã đi đến quyết định thiết kế một công cụ nhỏ, mục đích kéo tất cả nội dung cần thiết của Google Skills về máy nội bộ, đóng gói thành các file thuần văn bản Markdown, trong một kho (vault) của Obsidian; vừa xây dựng cho mình một Kho Kiến Thức Cá Nhân (Personal Knowledge Base); vừa tạo tiền đề cho các sự mở rộng/liên kết kiến thức thậm chí bên ngoài Google Skills; hoặc/và đóng gói thành các văn bản dạng sách A4/A5, tiện lợi cho việc tra khảo hoặc đọc trên các thiết bị cầm tay (iPad/Kindle) cá nhân.

```
* commit 1c822f5eb80d5e6350f1e170379b8cabcb3af657
| Author: samdx <18287802+samdx@users.noreply.github.com>
| Date:   Sun Jan 5 17:07:57 2025 +0800
|
|     Init the repo.
|
* commit 9eb4e75a4ffa4c01cee6c36eadc2f8ce282747fe
  Author: samdx <18287802+samdx@users.noreply.github.com>
  Date:   Sun Jan 5 17:06:54 2025 +0800

      Create .gitattributes
```

Mục tiêu đầu tiên của công cụ ban đầu hoàn toàn không có ý tưởng về việc xây dựng một Bộ Não Thứ 2 (https://fortelabs.com/blog/basboverview/), tuy nhiên có một sự vô tình trong cách triển khai, mà sau này khi OKF (Open Knowledge Format) xuất hiện, mô hình nội dung trong công cụ đã tương thích gần như hoàn toàn.

Tuy nhiên, báo cáo này chỉ tập trung vào phần kỹ thuật của công cụ, các nội dung khác liên quan về cách tổ chức kho PKB sẽ không được đề cập ở đây hoặc chỉ được nhắc qua, ví dụ: backlink hoặc graph view trong Obsidian giúp trực quan hóa mối quan hệ giữa các khóa học/môn học; cách tổ chức kho theo mô hình PARA và cách khai thác, vv..

Hi vọng với các nội dung dưới đây, sẽ đem đến một vài tham khảo có ý nghĩa cho các bạn sinh viên cùng lớp, cũng như biểu hiện được các nội dung kiến thức đã được Giảng Viên truyền đạt trong môn học IE221 tại UIT.

Lời Cảm Ơn

Đầu tiên, sinh viên thực hiện xin cảm ơn sự hướng dẫn của Giảng Viên bộ môn IE221 – *Thạc Sĩ Nghi Hoàng Khoa*, đã nhiệt tình hướng dẫn, hỗ trợ tài liệu, công cụ và các bước thực hiện trong quá trình thực hiện Đồ Án này.

Tiếp theo, sinh viên thực hiện xin gửi lời cảm ơn tới trung tâm CITD – UIT, đã tạo môi trường cho các bạn sinh viên được tham gia học tập và sinh hoạt trong không khí của UIT.

Cuối cùng, xin gửi lời cảm ơn tới Google Skills, hay chính xác hơn trước đó là Google Skills Boost đã có một môi trường mở với lượng kiến thức chất lượng để sinh viên có thể tham khảo (mặc dù bây giờ đã đóng hơn rất nhiều) và từ đó phát sinh ý tưởng cho công cụ này.

Nhận Xét Của Giảng Viên

Ngày/Tháng

Giảng Viên

Mục Lục

Lời Nói Đầu i
Lời Cảm Ơn iii
Nhận Xét Của Giảng Viên iv
Mục Lục v
Danh Sách Bảng vii
Danh Sách Hình Ảnh viii
Danh Sách Mã Nguồn ix
Bảng Viết Tắt xi
1. MÔ TẢ BÀI TOÁN 1
    1.1. Bối Cảnh 1
    1.2. Vấn Đề Cần Giải Quyết 2
    1.3. Mục Tiêu Đề Tài 2
    1.4. Phạm Vi Ứng Dụng 3
    1.5. Yêu Cầu 3
    1.6. Kết Chương 4
2. TRIẾT LÝ VÀ THIẾT KẾ 5
    2.1. Một Ứng Dụng, Không Phải Một Hệ Thống 5
    2.2. Con Người Trong Vòng Lặp 6
    2.3. Vault Như Bộ Não Thứ Hai 7
    2.4. Đối Chiếu Với Open Knowledge Format 8
    2.5. Kiến Trúc Gói 9
    2.6. Hai Quyết Định Không Dùng 10
    2.7. Kết Chương 10
3. CÀI ĐẶT 11
    3.1. Môi Trường Phát Triển 11
    3.2. Package Và Module 12
    3.3. Mô Hình Hướng Đối Tượng 13
    3.4. Kiểu Dữ Liệu, Hàm Và Cấu Trúc Điều Khiển 20
    3.5. Xử Lý Ngoại Lệ 23
    3.6. Phiên Đăng Nhập Và Thu Thập 24
    3.7. Lưu Trữ 26
    3.8. Kết Xuất Markdown 28
    3.9. Hai Runner 29
    3.10. Kết Chương 33
4. KIỂM THỬ, VẬN HÀNH VÀ BẢO TRÌ 34
    4.1. Kiểm Thử Offline 34

4.2. Chạy Lại Và Tính Idempotent 35
4.3. Khi Nền Tảng Thay Đổi 36
4.4. Cấu Hình Và Tính Di Động 36
4.5. Kết Chương 37
5. KẾT LUẬN 38
5.1. Kết Quả Đạt Được 38
5.2. Hạn Chế 40
5.3. Hướng Phát Triển 40
5.4. Lời Kết 40
A Nguồn Gốc Và Tư Liệu 42
A.1 Nguồn Gốc Mã Nguồn 42
A.2 Danh Sách Commit Refactor 42
A.3 Bảng Port Từ Dòng Chính 43
A.4 Kịch Bản Demo 44
A.5 Chạy Từ Bản Nộp 44
A.6 Công Cụ Sử Dụng 44
Tài Liệu Tham Khảo 45

Danh Sách Bảng

Bảng 1 Sáu cấp dữ liệu của Google Skills 1
Bảng 2 Bốn câu hỏi của người học: trên site và trong vault 2
Bảng 3 Yêu cầu chức năng 3
Bảng 4 Yêu cầu phi chức năng 4
Bảng 5 Quy mô hai portal theo danh mục đã kéo về (09/2026) 7
Bảng 6 Đối chiếu khoá frontmatter của vault với OKF v0.2 8
Bảng 7 Các lệnh của `skills-scraper`, theo `--help` thật 30
Bảng 8 Bộ test offline, 15 test trong 4 tệp 34
Bảng 9 Site đổi, mã đổi theo: năm thay đổi thật và commit xử lý 36
Bảng 10 Trạng thái 11 yêu cầu chức năng và 6 yêu cầu phi chức năng 38
Bảng 11 Mười chương của IE221 và nơi chúng xuất hiện trong mã 39
Bảng 12 Mười ba việc port từ dòng chính và commit thực hiện 43
Bảng 13 Kịch bản demo năm bước 44

Danh Sách Hình Ảnh

Hình ảnh 1 Graph view của vault trong Obsidian: mỗi nút là một note, mỗi cạnh là một liên kết tương đối mà note tự ghi ra 8
Hình ảnh 2 Phụ thuộc một chiều: runner gọi `model`, `model` gọi `services`, `services` làm việc với file và trình duyệt 9
Hình ảnh 3 Sơ đồ lớp của `model/`: hai nhánh kế thừa từ `Serialize` 13
Hình ảnh 4 Hai cây kế thừa như IDE nhìn thấy trong mã thật 18
Hình ảnh 5 `skills-scraper` chạy trong terminal 31
Hình ảnh 6 `skills-scraper-tui`: cùng lõi, giao diện khác 33

Danh Sách Mã Nguồn

Mã nguồn 1 Cây phân cấp, các loại *activity* do Google quyết định 2
Mã nguồn 2 Frontmatter của note course 892 ở v2.0.0 9
Mã nguồn 3 `pyproject.toml`: tên, phiên bản, Python tối thiểu và các thư viện lúc chạy 11
Mã nguồn 4 `pyproject.toml`: hai entry point và build backend 12
Mã nguồn 5 `__init__.py`: phiên bản đọc từ metadata của gói, không chép tay 12
Mã nguồn 6 Cây package `src/skills_scraper/` 13
Mã nguồn 7 `model/serialize.py`: gốc của cả hai cây kế thừa 14
Mã nguồn 8 `BaseEntity.__init__`: ba thuộc tính định danh 14
Mã nguồn 9 `BaseEntity.type` và `BaseEntity.url`: suy ra từ tên lớp 15
Mã nguồn 10 `BaseEntity.generate_markdown`: bản mặc định chỉ có frontmatter 16
Mã nguồn 11 `BaseEntity.save_markdown`: gọi `generate_markdown` của lớp con đang chạy 16
Mã nguồn 12 `model/lab.py`: lớp con kế thừa `BaseEntity` và ghi đè `generate_markdown` 17
Mã nguồn 13 `model/courses.py`: lớp con chỉ khai `API_URL`, phần còn lại kế thừa 18
Mã nguồn 14 `BaseEntity.to_dict`: cái gì được lưu ra đĩa 19
Mã nguồn 15 `Course.process_lab`: tạo `Lab`, nạp JSON, bỏ qua nếu đã có . 19
Mã nguồn 16 `Course.process_lab`: lưu lab vừa tải 19
Mã nguồn 17 `Course.process_lab`: cập nhật tập hợp `Labs` 19
Mã nguồn 18 `yaml_scalar`, `yaml_list`: chuỗi thành YAML an toàn 20
Mã nguồn 19 `numeric_id`: tuple làm khoá sắp xếp, $9 < 10 < 108$ 20
Mã nguồn 20 `Course.process_step`: bảng dispatch theo loại activity 21
Mã nguồn 21 `Course.process_step`: loại lạ được giữ nguyên và báo 21
Mã nguồn 22 `Course._parse_lesson_item`: đệ quy trên cây item 22
Mã nguồn 23 `Collection.fetch_catalog`: chữ ký và tham số mặc định 22
Mã nguồn 24 `Collection.fetch_catalog`: vòng lặp phân trang có cầu chì . . 22
Mã nguồn 25 `store.write_text`: ghi nguyên tử với `try/except/raise` 23
Mã nguồn 26 `browser.ensure_authenticated`: bắt `KeyboardInterrupt`, `EOFError` 24
Mã nguồn 27 `BaseEntity.save_json`: ghi trước, cập nhật chỉ mục sau 24
Mã nguồn 28 `browser.launch_browser`: tra bảng trình duyệt, cờ dùng chung, profile cố định 25

Mã nguồn 29 `browser.open_page`, `browser.get_page`: cửa duy nhất tới trang web cho `model/` 25
Mã nguồn 30 `Course.extract_course_metadata`: bỏ qua theo `datePublished`, tường minh 26
Mã nguồn 31 Thư mục `data/`: một thực thể, một file 27
Mã nguồn 32 `Index.upsert` và `Index.rebuild`: chỉ mục luôn dựng lại được từ file 27
Mã nguồn 33 `BaseEntity.generate_front_matter`: khoá OKF, khoá riêng và alias cũ 28
Mã nguồn 34 Note `courses/Conversational-AI-on-Vertex-AI-and-Dialogflow-CX.md`, rút gọn 29
Mã nguồn 35 `cli.main`: sub-parser `list` với hai nhóm loại trừ lẫn nhau ... 30
Mã nguồn 36 `cli.cmd_reindex`: một lệnh CLI là một hàm gọi vào `services` 31
Mã nguồn 37 Ghép CLI với `jq`, `while` của shell và `git` 31
Mã nguồn 38 `CloudSkillsBoost.driver`: trình duyệt khởi tạo lười, dùng chung cho cả phiên 32
Mã nguồn 39 `tui.main`: `try/finally` đảm bảo đóng trình duyệt khi thoát . 32
Mã nguồn 40 `uv sync` và `uv run pytest` từ clone sạch: 15/15, không cần mạng 35
Mã nguồn 41 `config.load_settings`: trộn ba lớp, đường dẫn tương đối tính từ `config.yaml` 37
Mã nguồn 42 `git log --oneline v2.0.0..refactor-ie221`, 22 commit 43
Mã nguồn 43 Các lệnh để chạy từ bản zip nộp kèm 44

Bảng Viết Tắt

Viết Tắt	Nghĩa Đầy Đủ
ANSI	American National Standards Institute (mã màu terminal)
API	Application Programming Interface
CLI	Command Line Interface (giao diện dòng lệnh)
CODE	Capture, Organize, Distill, Express (khung Bộ Não Thứ Hai)
CSB	Cloud Skills Boost (tên cũ của Google Skills)
CSV	Comma-Separated Values
CTA	Cloud Technical Architect
DOM	Document Object Model
FR	Functional Requirement (yêu cầu chức năng)
GCP	Google Cloud Platform
GUI	Graphical User Interface (giao diện đồ hoạ)
HTML	HyperText Markup Language
JSON	JavaScript Object Notation
KISS	Keep It Simple Stupid
LLM	Large Language Model (mô hình ngôn ngữ lớn)
NFR	Non-Functional Requirement (yêu cầu phi chức năng)
OKF	Open Knowledge Format (Google)
OOP	Object-Oriented Programming (lập trình hướng đối tượng)
PARA	Projects, Areas, Resources, Archives (cách tổ chức kho ghi chú)
PDF	Portable Document Format
PEP	Python Enhancement Proposal
PKB	Personal Knowledge Base (kho kiến thức cá nhân)
SSOT	Single Source Of Truth (nguồn sự thật duy nhất)
TOC	Table Of Contents (mục lục)
TUI	Text User Interface (giao diện tương tác trong terminal)
URL	Uniform Resource Locator
UTF-8	Unicode Transformation Format 8-bit
YAML	YAML Ain't Markup Language

CHƯƠNG 1.

MÔ TẢ BÀI TOÁN

Chương này đặt bài toán: Google Skills là gì và đã thay đổi thế nào trong hai năm gần đây, nội dung trên đó được phân cấp ra sao, người học thiếu gì khi chỉ có một trang web, và từ đó công cụ cần làm gì (yêu cầu chức năng) và phải có tính chất gì (yêu cầu phi chức năng). Phạm vi được khoanh rõ ngay từ đầu, bao gồm cả những thứ cố ý không làm, để các chương sau không phải giải thích lại.

1.1. Bối Cảnh

1.1.1. Google Skills và những thay đổi gần đây

Như đã đề cập, Google Skills (skills.google), trước đây là Google Cloud Skills Boost, là nền tảng đào tạo chính thức của Google Cloud, gồm hàng trăm lộ trình học (path), hàng nghìn khoá học (course) và bài thực hành (lab).

Trong khoảng hai năm, từ cuối 2024 đến đầu 2026, nền tảng đã có những thay đổi trong cả định hướng hoạt động và trong bộ nội dung được triển khai, và từ đó ảnh hưởng trực tiếp tới cách thu thập nội dung, ví dụ: từ một nền tảng mở, Google Skills bây giờ đã yêu cầu phải có tài khoản đăng nhập để có thể xem nội dung môn học, từ đó một hệ quả kỹ thuật phát sinh: thư viện `requests` không còn đủ, việc thu thập phải đi qua một phiên trình duyệt đã đăng nhập (Selenium), vì vậy công cụ sẽ cần thực hiện thêm việc quản lý phiên thay vì chỉ kéo nội dung như trước.

1.1.2. Phân Cấp Dữ Liệu

Cấp	Thực thể	Ghi chú
1	**Portal**	Bao gồm *public* và *partner*; báo cáo này chỉ đề cập *public* portal vì *partner* là dành cho đối tác của Google, và cần có tài khoản riêng.
2	**Path**	Lộ Trình, chứa nhiều Course và Lab theo thứ tự.
3	**Course**	Khoá Học, có mô tả, mục tiêu, chủ đề và từng chương.
4	**Module**	Chương của khoá học, sẽ có từng bài học cụ thể.
5	**Step**	Các bước trong chương.
6	**Activity**	Các bài học, có thể là video, lab, quiz, pdf hoặc html, hoặc loại nội dung bất kỳ nào khác.

Bảng 1: Sáu cấp dữ liệu của Google Skills

```
Portal (public | partner)
└ Path (learning path)
  └ Course
    └ Module
      └ Step
        └ Activity
          └ video | lab        | quiz        |
          └ link  | document   | html_bundle |
          └ badge | credential | ...
```

Mã nguồn 1: Cây phân cấp, các loại *activity* do Google quyết định

> **Ghi Chú**
>
> - Lab vừa là một hoạt động trong Course vừa là một thực thể độc lập, đôi khi được tham chiếu trực tiếp bởi Path, điều này đôi lúc tạo nên sự phức tạp trong logic của việc liên kết các thực thể.
> - Tập các loại hoạt động (**Activity**) thuộc về Google chứ không thuộc về ứng dụng: Google Skills có thể thay đổi các loại hoạt động bất kỳ lúc nào, hiện tại thường là sẽ thêm mới dựa theo cách triển khai nội dung của các khóa học/ môn học mới.

1.2. Vấn Đề Cần Giải Quyết

Nền tảng cho xem một trang tại một thời điểm. Bốn câu hỏi người học hay đặt ra nhất lại không trả lời được trên bản thân trang độc lập đó, trong khi chúng là tính năng có sẵn của Obsidian khi nội dung là Markdown liên kết:

Câu hỏi	Trên skills.google	Trong vault Obsidian
Course này **thuộc những path** nào?	phải tìm ngược từng path	backlink của note
Hai path **chồng lấn bao nhiêu** course?	không có	graph view, truy vấn
Transcript nào **nhắc tới một khái niệm**?	khó tìm trong transcript	tìm toàn văn, offline
Course này **thay đổi gì** từ lần học trước?	không có lịch sử	`date_published` và `git diff`

Bảng 2: Bốn câu hỏi của người học: trên site và trong vault

Quy trình hiện tại của người học trên Google Skills: duyệt danh mục, mở từng khoá, xem video, ghi chú tay. Nội dung nằm rải rác qua video, quiz, lab, tài liệu; không có bản xuất, không có bản offline, không tìm kiếm xuyên khoá. Có một sự rời rạc trong một hệ thống được thiết kế trông rất chặt chẽ.

1.3. Mục Tiêu Đề Tài

Mục tiêu đầu tiên, công cụ thực tiễn:

- Xây dựng một ứng dụng/công cụ Python dòng lệnh.
- Thu thập Path, Course, Lab từ skills.google vào kho JSON (file-based database).
- Sinh ra một vault Markdown dùng được ngay trong Obsidian
- Cung cấp hai giao diện terminal (CLI cho tự động hoá, TUI cho tương tác).

Mục tiêu thứ hai, về mặt học thuật, là dùng chính ứng dụng này làm chứng cứ cho các kiến thức của môn học: **module** và **package**, **lớp** và **đối tượng**, **kế thừa**, xử lý ngoại lệ, vv...

1.4. Phạm Vi Ứng Dụng

1.4.1. Bao Gồm

- Public portal;
- Ba loại thực thể chính: Path, Course, Lab;
- Activity: transcript video, quiz, lab, link, tài liệu PDF, html_bundle;
- Kết xuất JSON và Markdown;
- Hai runner terminal: CLI, TUI (Terminal User Interface).

1.4.2. Không Bao Gồm

- Partner Portal (sinh viên không còn quyền truy cập);
- Chạy Lab;
- Quản lý tài khoản, mua khoá học;
- Giao diện đồ hoạ và Cơ Sở Dữ Liệu (lý do sẽ được trình bày ở Chương 2 (trang 5).

1.5. Yêu Cầu

1.5.1. Yêu Cầu Chức Năng

Mã	Yêu cầu
FR-01	Liệt kê danh mục `path`, `course`, `lab`; làm mới từ site hoặc đọc offline
FR-02	Thu thập một `path` và lan xuống các `course`, `lab` của nó
FR-03	Thu thập một `course` với đầy đủ `module`, `step`, `activity`
FR-04	Thu thập một `lab` độc lập
FR-05	Bỏ qua course có `datePublished` không đổi, trừ khi `--force`
FR-06	Sinh Markdown cho `path`, `course`, `lab`; tuỳ chọn `--toc`, `--no-transcript`
FR-07	Tìm kiếm theo tên (`name`), `id`, chủ đề (`topic`)
FR-08	Dựng lại chỉ mục từ dữ liệu JSON (`reindex`)
FR-09	Tải tài liệu PDF vào `materials/courses/<id>/`
FR-10	Sinh tệp prompt JSON cho mô hình ngôn ngữ lớn (LLM).
FR-11	Giao diện tương tác dạng menu (TUI)

Bảng 3: Yêu cầu chức năng

> **Note**
>
> FR-10: Sinh tệp prompt JSON cho mô hình ngôn ngữ lớn (LLM) để tóm tắt nội dung transcript, đã từng được đề xuất, nhưng giờ tạm ngừng xây dựng.

1.5.2. Yêu Cầu Phi Chức Năng

Mã	Yêu cầu
NFR-01	Không trạng thái (stateless): mỗi lần chạy đọc tệp, ghi tệp; chạy lại là *idempotent* (nguyên bản)
NFR-02	Dữ liệu và đầu ra là văn bản thuần (plain text), UTF-8, đọc được bằng `git diff`
NFR-03	Cấu hình tường minh qua biến môi trường và tệp văn bản (`config.yaml`)
NFR-04	Lỗi tường minh: một `activity` hỏng không làm hỏng cả `course` , mọi ngoại lệ đều được báo
NFR-05	Ghi tệp nguyên tử (`atomic`): nếu gặp ngắt giữa chừng không để lại nửa tệp lỗi
NFR-06	Môi trường tái lập: `uv sync` từ một bản clone sạch là có thể bắt đầu

Bảng 4: Yêu cầu phi chức năng

1.6. Kết Chương

Chương này mô tả bài toán thực tế của sinh viên: nội dung học tập nằm trên một nền tảng chỉ cho xem từng trang, ngày càng đóng, và không trả lời được những câu hỏi liên kết giữa các khoá học. Nhu cầu phát sinh từ công việc, nên lời giải là một công cụ thực tế với mười một yêu cầu chức năng (Bảng 3) và sáu yêu cầu phi chức năng (Bảng 4), có những đặc thù nhất định và không nhất thiết dùng hết mọi kiến thức của môn học. Chương 2 trình bày triết lý và các quyết định thiết kế để trả lời bài toán đó bằng một ứng dụng nhỏ.

CHƯƠNG 2.

TRIẾT LÝ VÀ THIẾT KẾ

Trước khi đi vào mã nguồn, chương này trình bày cách nghĩ đã quyết định hình hài của công cụ: vì sao nó là một ứng dụng nhỏ chứ không phải một hệ thống, vì sao con người vẫn ở trong vòng lặp khi site thay đổi, vault sinh ra đóng vai trò gì trong mô hình Bộ Não Thứ Hai và khớp với Open Knowledge Format của Google ở đâu, kiến trúc gói được chia thế nào, và hai quyết định cố ý không dùng (cơ sở dữ liệu, giao diện đồ hoạ) dù đó là hai chương của môn học.

2.1. Một Ứng Dụng, Không Phải Một Hệ Thống

Công cụ được thiết kế dưới góc nhìn của kỹ sư hệ thống và theo triết lý Unix *[1]*: một thành phần nhỏ, đúng một việc, ghép được với các công cụ khác, mà không phải là một "hệ thống" hay một nền tảng hoàn toàn độc lập. Năm nguyên tắc dưới đây quyết định mọi lựa chọn trong các chương sau.

2.1.1. Làm Một Việc Và Làm Tốt

> *KISS*: **Keep It Simple, Stupid**
>
> - Làm một việc, và làm thật tốt việc đó.
> - Hoàn toàn có thể kết hợp với các công cụ khác để tạo thành một luồng công việc.
> - Dễ dàng được thay thế mà không ảnh hưởng đến tổng thể.

Ứng dụng chỉ thu thập và chuẩn hoá.

- Lệnh `fetch` đưa nội dung về JSON.
- Lệnh `md` chuyển JSON thành Markdown, hai lệnh không (cần) biết nhau.
- Ba thư viện ngoài lúc chạy (selenium, beautifulsoup4, requests).
- Không daemon, không server.

2.1.2. Đơn Giản Để Luôn Thay Thế Được

Thứ bền nhất không phải công cụ mà là dữ liệu nó để lại. JSON và Markdown thuần vẫn đọc được khi công cụ không còn tồn tại.

Dự án đã tự chứng minh: dòng chính của repo thay lõi Python bằng lõi Go (thử nghiệm) mà vault không đổi định dạng. Một công cụ khoá dữ liệu của người dùng vào chính nó thì không phải là khó thay thế, mà biến dữ liệu thành con tin.

> **Tiêu Chí**
>
> - Đơn giản đến mức có thể được thay thế bất kỳ lúc nào.
> - Hiệu quả đến mức không cần phải thay thế.

2.1.3. Là Một Phần Của Hệ Thống

Ứng dụng giao tiếp bằng tệp văn bản và mã thoát, nên ghép được với `grep`, `jq`, `git`, `cron`, Obsidian và các plugin AI mà không cần API riêng. Đồ thị khoảng 3.600 note trong vault không được thiết kế trước: nó hình thành từ các liên kết tương đối mà mỗi note tự ghi ra.

> **Emergence**
>
> Nguyên Lý Đột Sinh (Emergence) – một phần "ngốc nghếch" của một hệ thống *thông minh*.

2.1.4. Tường Minh

- Cấu hình nằm trong biến môi trường và tệp `config.yaml` văn bản thuần, có tệp mẫu trong repo.
- Mỗi note Markdown tự khai nó được sinh bởi phiên bản nào, từ trang nào, trang đó sửa lần cuối khi nào đều được nêu rõ tường minh.
- Khi bỏ qua một `course` vì course đó không đổi, ứng dụng nêu rõ ra là đã bỏ qua và vì lý do gì.

2.1.5. Thêm Hay Không Thêm

Mỗi thư viện và mỗi tầng hạ tầng phải trả lời được câu hỏi "nó đóng góp được gì".

Quy ước của dự án: một thư viện chỉ được khai báo trong đúng *commit* đầu tiên `import` nó. Cơ sở dữ liệu lớn chỉ xứng đáng khi cần hiệu năng cao, ghi đồng thời song song, hoặc xử lý phân tán; bài toán này không cần cả ba, vì vậy đây là lý do báo cáo này không sử dụng một số kiến thức nhất định trong môn học.

2.2. Con Người Trong Vòng Lặp

- Câu hỏi: *nếu Google thay đổi cấu trúc website hay khoá học, ứng dụng có tự phát hiện và thay đổi theo không?*
- Câu trả lời tách làm hai vế: phát hiện thì **có**, tự thích nghi thì **không**, và đó là chủ ý thiết kế của người thực hiện.

Về phát hiện: loại *activity* lạ được giữ nguyên dữ liệu thô và ghi log:

- Thiếu metadata hoặc đề cương thì dừng course đó và báo rõ;
- Phiên đăng nhập hết hạn thì dừng chờ người dùng;
- Bộ mẫu HTML trong `tests/` sẽ kích hoạt một thay đổi markup (của Google) thành một test thất bại (và từ đó cần xử lý tương ứng).

Về không tự thích nghi, có ba lý do.

- Một scraper tự sửa selector thực chất là **đoán**, và đoán sai sẽ ghi dữ liệu sai vào vault một cách âm thầm; *lỗi im lặng nguy hiểm hơn lỗi ồn ào*, thà để ồn ào để biết đó là gì, còn hơn để mọi thứ sai trong im lặng, đây cũng là chủ ý thiết kế.
- Có đi theo thay đổi hay không là quyết định về giá trị: nếu mục tiêu cốt lõi (lấy nội dung về) vẫn đạt, thời gian, công sức và tài nguyên tính toán bỏ ra có xứng đáng không; bản thân logic của công cụ không trả lời được câu đó, và đó là lúc con người tham gia trong "vòng lặp".

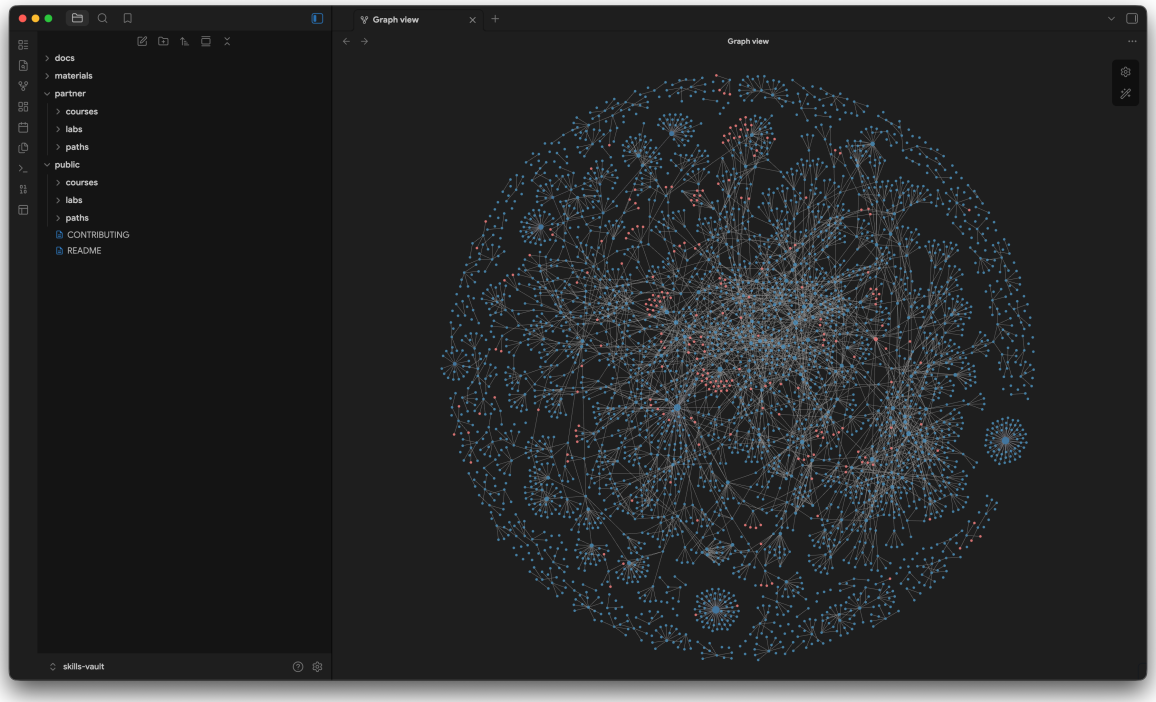
Lịch sử dự án đã kiểm chứng cách làm này (chương 4): mỗi thay đổi của site là một commit có nguồn và có lý do. Mô hình triển khai: công cụ báo lỗi, người quyết định, git ghi lại lịch sử bao gồm cả lý do, cách xử lý, vv... thành một nguồn sự thật duy nhất (SSOT – Single Source Of Truth).

2.3. Vault Như Bộ Não Thứ Hai

Theo khung CODE (Capture, Organize, Distill, Express) của Tiago Forte *[2]*, ứng dụng làm **Capture** và một phần **Distill** (`--toc` , `--no-transcript` , tệp prompt). **Organize** thuộc về Obsidian, **Express** thuộc về người học. Ứng dụng cố ý và nên dừng ở mức đó.

Portal	Paths	Courses	Labs
public	73	416	824
partner	181	899	1209

Bảng 5: Quy mô hai portal theo danh mục đã kéo về (09/2026)



Hình ảnh 1: Graph view của vault trong Obsidian: mỗi nút là một note, mỗi cạnh là một liên kết tương đối mà note tự ghi ra

2.4. Đối Chiếu Với Open Knowledge Format

OKF v0.2 *[3]* chỉ bắt buộc mọi tệp `.md` có frontmatter YAML đọc được với khoá `type` không được rỗng. Vault đã thoả điều này từ 2025 vì `type` được suy ra từ tên lớp. Việc trùng khớp không phải ngẫu nhiên: cả hai cùng xuất phát từ ràng buộc "một khái niệm là một tệp, siêu dữ liệu (metadata) ở đầu, thân bài cho người đọc".

> **Fun Fact**
>
> Thực ra cách trình bày file Markdown này là kế thừa từ thời sử dụng Jekyll.

Vault v2.0.0	OKF v0.2	Trạng thái
`type`	`type` (bắt buộc)	đã có
`title`	`title`	đã có
`url`	`resource`	khác tên, giữ làm alias
`topics`	`tags`	khác tên, giữ làm alias
mô tả trong thân bài	`description`	bổ sung
`scraped_date`	`generated {by, at}`	bổ sung
`date_published`	`sources[].last_modified`	bổ sung
không có	`index.md` , `log.md`	hướng phát triển

Bảng 6: Đối chiếu khoá frontmatter của vault với OKF v0.2

Frontmatter của một note ở v2.0.0, trước khi đối chiếu; bản sau khi mở rộng được trình bày ở Phần 3.8:

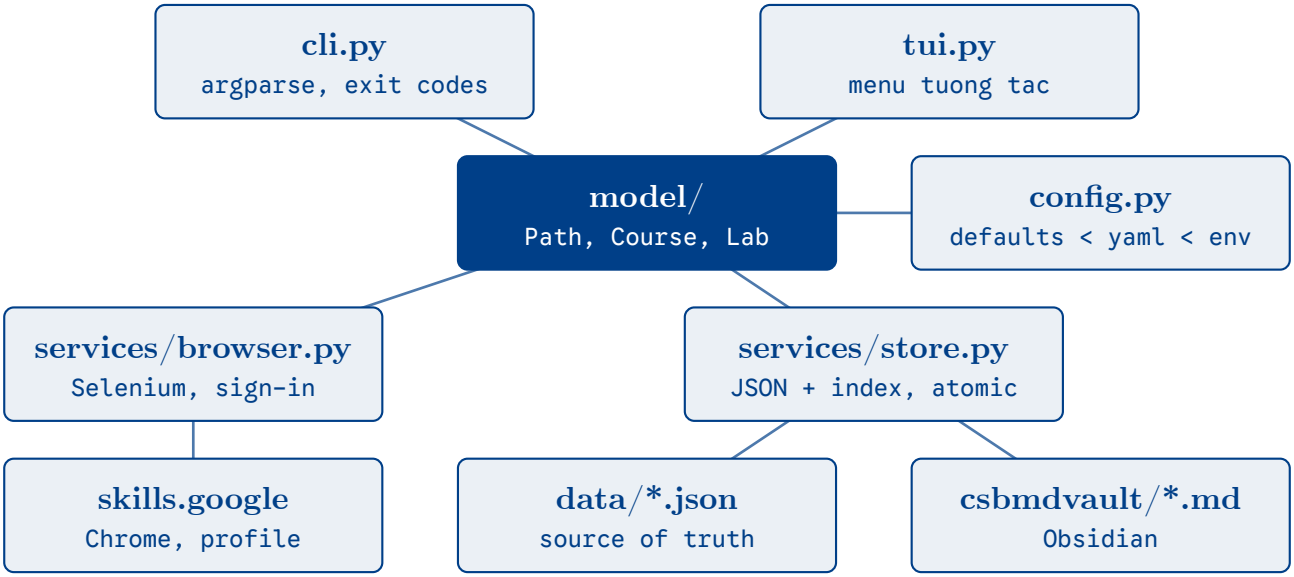
```
"id": "892"
name: "Conversational AI on Vertex AI and Dialogflow CX"
type: "Course"
url: "https://www.cloudskillsboost.google/course_templates/892"
description: "In this course you will learn how to use the new
generative AI features..."
date: "2025-01-06"
datePublished: "2023-11-21",
```

Mã nguồn 2: Frontmatter của note course 892 ở v2.0.0

2.5. Kiến Trúc Gói

```
src/skills_scraper/
  __init__.py
  config.py     cấu hình, URL, selector
  cli.py        runner dòng lệnh
  tui.py        runner tương tác
  model/        lõi: Serialize, BaseEntity, Collection và các lớp con
  services/     browser.py (phiên đăng nhập), store.py (lưu trữ)
  utils/        xử lý văn bản, ghi tệp nguyên tử
```

Hướng phụ thuộc một chiều: runner gọi `model/`, `model/` dùng `services/` và `utils/`. Toàn bộ kiến thức về skills.google nằm trong `config.py` và các phương thức phân tích trang; mô hình, lưu trữ, kết xuất và giao diện không biết Google.



Hình ảnh 2: Phụ thuộc một chiều: runner gọi `model`, `model` gọi `services`, `services` làm việc với file và trình duyệt

2.6. Hai Quyết Định Không Dùng

2.6.1. Không Cơ Sở Dữ Liệu (Chương 10 - IE221)

- Kho có dưới 5.000 bản ghi, Tìm Kiếm Tuyến Tính có thể trả về kết quả trong mili giây.
- Tệp phẳng (flat-file) và văn bản thuần (plain text), nên đọc được bằng git và Obsidian hoặc bất kỳ công cụ nào khác.
- Không trạng thái (stateless) nên chạy ở đâu cũng được.

> **Ghi Chú**
>
> Phiên bản v2.0.0 từng dùng song song JSON và TinyDB, và hiện đã bỏ TinyDB ở các phiên bản sau, vì không xứng đáng để bảo trì một lúc hai nguồn sự thật (flat-file trong git và database trong TinyDB) trong khi không đem lại giá trị quá khác biệt.

2.6.2. Không Giao Diện Đồ Hoạ (Chương 09 – IE221)

- Người dùng và môi trường mục tiêu là người làm việc trong terminal.
- Giao diện đồ hoạ thật đã tồn tại ở nhánh Tauri (như một ví dụ về khả năng mở rộng) nhưng sẽ không được ghép vào dòng/chức năng chính.
- Báo cáo này sẽ chỉ tập trung vào phần lõi của công cụ (model) nơi biểu hiện rõ nhất các nội dung của chương trình học.

> **Ghi Chú**
>
> Đây đều là các quyết định thiết kế có lý do và chủ đích, mà không phải là sự vô tình bỏ sót kỹ năng đã học hoặc nội dung chương trình

.

2.7. Kết Chương

Chương này trình bày các lý do và quyết định trong quá trình xây dựng công cụ: năm nguyên tắc của một ứng dụng nhỏ, vai trò của con người khi site thay đổi, vị trí của vault trong khung CODE và sự trùng khớp với OKF (Bảng 6), kiến trúc gói phụ thuộc một chiều (Hình ảnh 2), cùng những phần CÓ và KHÔNG với nguyên do tương ứng.

Đây là tiền đề cho Chương 3, nơi các nội dung cốt lõi của môn học được chỉ ra trên chính mã nguồn đã chọn theo các nguyên tắc ở đây.

CHƯƠNG 3.

CÀI ĐẶT

Chương này đi vào mã nguồn theo đúng thứ tự các chương của môn học: môi trường và gói (chương 01, 06), mô hình hướng đối tượng (chương 08), kiểu dữ liệu, rẽ nhánh, vòng lặp và hàm (chương 02 tới 05), xử lý ngoại lệ (chương 07); rồi tới ba mảng mà bất kỳ công cụ thu thập nào cũng phải có: phiên đăng nhập và thu thập, lưu trữ, kết xuất Markdown; và cuối cùng là hai runner trên cùng một lõi. Mọi đoạn mã trong chương được `tools/snippets.py` bóc bằng `ast` từ branch `refactor-ie221` theo tên symbol, không gõ tay, nên không thể có đoạn mã nào trên giấy mà không có trong repo.

3.1. Môi Trường Phát Triển

`uv` *[4]* được dùng để khai báo, quản lý gói và khoá phiên bản:

- Nhất quán, có thể tái lập ở một môi trường mới bất kỳ đâu: `uv sync` từ một bản clone sạch tải đúng Python 3.12 và đúng các phiên bản trong `uv.lock`.
- Một công cụ duy nhất, `uv`, với một định dạng đánh dấu (markup) thân thiện, `toml`.

`pyproject.toml` *[5]* khai báo tên và phiên bản của công cụ, bốn thư viện lúc chạy (selenium, beautifulsoup4, requests, pyyaml), và hai *runner* dưới dạng console script, tức hai lệnh `skills-scraper` và `skills-scraper-tui` xuất hiện trong `PATH` sau khi cài.

```toml
[project]
name = "skills-scraper"
version = "2.0.1"
description = "Scrape Google Skills paths, courses and labs into JSON
and an Obsidian-ready Markdown vault."
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "selenium>=4.40",
    "beautifulsoup4>=4.12",
    "requests>=2.32",
    "pyyaml>=6.0.3",
```

Mã nguồn 3: `pyproject.toml`: tên, phiên bản, Python tối thiểu và các thư viện lúc chạy

```toml

[project.scripts]
skills-scraper = "skills_scraper.cli:main"
skills-scraper-tui = "skills_scraper.tui:main"

[build-system]
requires = ["uv_build>=0.8,<0.13"]
build-backend = "uv_build"
```

Mã nguồn 4: `pyproject.toml`: hai entry point và build backend

`.python-version` cho `uv` biết phiên bản Python được dùng; `__init__.py` của gói đọc lại phiên bản từ metadata đã cài, nên con số chỉ được khai ở một chỗ.

```python
"""Google Skills Scraper: paths, courses and labs from skills.google into JSON
and Markdown."""

from importlib.metadata import version

__version__ = version("skills-scraper")
```

Mã nguồn 5: `__init__.py`: phiên bản đọc từ metadata của gói, không chép tay

3.2. Package Và Module

Mã nằm trong một package `skills_scraper` dưới `src/`, cách bố trí được Python Packaging User Guide khuyến nghị để mã chỉ chạy được khi đã cài, không chạy nhầm từ thư mục làm việc. Package có ba sub-package và ba module ở gốc:

- `model`: cốt lõi, các lớp thực thể và tập hợp; là nơi biểu hiện rõ nhất nội dung môn học.
- `services`: hai dịch vụ mà `model` cần: `browser.py` chạy trình duyệt, `store.py` ghi và đọc file.
- `utils`: tiện ích văn bản, ví dụ lọc ký tự không hợp lệ cho tên file và bỏ thẻ HTML.
- `config.py`, `cli.py`, `tui.py`: cấu hình và hai runner.

```
src/skills_scraper/
├── __init__.py           # __version__ đọc từ pyproject
├── config.py             # defaults < yaml < env
├── cli.py                # cli (Command Line Interface)
├── tui.py                # tui (Text User Interface)
├── model/                # LÕI QUAN TRỌNG
│   ├── __init__.py       # __init__
│   ├── base_entity.py
│   ├── collection.py
│   ├── path.py
│   ├── course.py
│   ├── courses.py
│   ├── lab.py
│   ├── labs.py
│   ├── paths.py
│   ├── serialize.py
│   └── topics.py
├── services/
│   ├── __init__.py       # __init__
│   ├── browser.py        # browser (Selenium)
│   └── store.py          # store (JSON, index, atomic)
└── utils/
    ├── __init__.py       # __init__
    └── utils.py

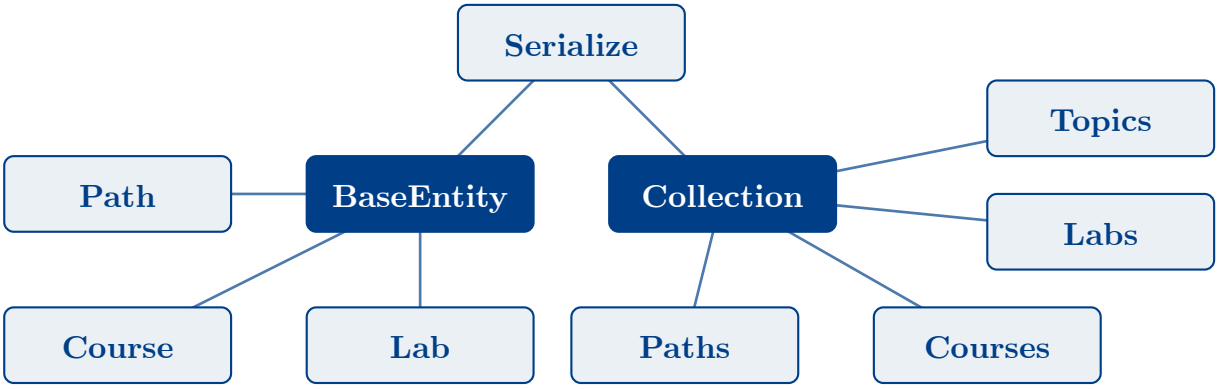
```

Mã nguồn 6: Cây package `src/skills_scraper/`

3.3. Mô Hình Hướng Đối Tượng

3.3.1. Hai Cây Kế Thừa

Hai nhánh kế thừa xuất phát từ cùng một điểm, `Serialize`, lớp biết biến một đối tượng thành dạng thân thiện với JSON.



Hình ảnh 3: Sơ đồ lớp của `model/`: hai nhánh kế thừa từ `Serialize`

Trong đó:

- Trái: **thực thể** x cụ thể, ở đây là Path, Course, Lab.
  - `BaseEntity` biết id, tên, mô tả, đường dẫn file và cách ghi JSON, Markdown. Lớp con chỉ thêm dữ liệu riêng và cách render.
- Phải: **danh sách của** x, ở đây là tập hợp của Path, Course, Lab.
  - `Collection` là `{id: name}` cộng `fetch_catalog` (kéo danh sách của một loại).
  - Lớp con chỉ khai `API_URL`.
- **Kết hợp**:
  - `Portal` cho biết các `Path` khả dụng (không triển khai ở bản IE221).
  - `Path` cho biết danh sách các khoá học (`Course`).
  - `Course` tạo `Lab` và cập nhật `Labs`.
  - `Topics` đọc thuộc tính của `Courses`, `Path`, `Lab`.

`Serialize` thực ra rất đơn giản:

- `to_dict`: chuyển đối tượng thành từ điển, vốn thân thiện với JSON.
- `to_json`: chuỗi JSON của từ điển đó, chức năng cốt lõi mà các lớp kế thừa dùng trực tiếp.

```python
import json

# Class to serialize objects to JSON
class Serialize:

    # Convert object to dictionary containing non-private attributes
    def to_dict(self):
        return {key.lstrip('_'): value for key, value in self.__dict__.items()}

    # Convert object to JSON string
    def to_json(self):
        return json.dumps(self.to_dict(), ensure_ascii=False, indent=2)

```

Mã nguồn 7: `model/serialize.py`: gốc của cả hai cây kế thừa

3.3.2. Lớp, Đối Tượng Và Thuộc Tính Suy Diễn

`BaseEntity.__init__()` khai báo các thuộc tính nguyên bản (attributes) để định danh một thực thể: `id`, `name`, `description`.

```python
    def __init__(self,
                 id: str,
                 name: str,
                 description: str):
        self.id = id
        self.name = name
        self.description = description

```

Mã nguồn 8: `BaseEntity.__init__`: ba thuộc tính định danh

Nhưng để tự nhận diện và phân loại các thực thể, `BaseEntity` khai báo các thuộc tính nội sinh (properties) với `@property`, nhờ đó việc kế thừa trở nên xuyên suốt hơn:

- `@property type` trả `self.__class__.__name__`.
- `@property url` tra bảng theo `type`, ném `ValueError` khi gặp loại lạ.

```python
    @property
    def type(self):
        """
        Dynamically determine the type based on the class name.
        """

        return self.__class__.__name__

    @property
    def url(self):
        """
        Dynamically generate the URL based on the type.
        """

        base_url = {
            "Path": BASE_URL_PATHS,
            "Course": BASE_URL_COURSES,
            "Lab": BASE_URL_LAB
        }.get(self.type, None)

        if not base_url:
            raise ValueError(f"Invalid entity type: {self.type}")

        return f"{base_url}/{self.id}"

```

Mã nguồn 9: `BaseEntity.type` và `BaseEntity.url`: suy ra từ tên lớp

> **Hệ Quả**
>
> Loại (`type`) của một thực thể không cần lưu sẵn mà tự suy ra, nên lớp con không thể thiếu nó; từ đó frontmatter theo chuẩn OKF luôn có `type` (gần như) miễn phí, không cần kiểm lại.

3.3.3. Kế Thừa Và Đa Hình

Kế thừa và đa hình xuất hiện ở nhiều nơi trong mã, rõ nhất là cặp `generate_markdown` và `save_markdown`: phương thức thứ nhất có bản mặc định ở lớp cha và được ghi đè (override) ở Path, Course, Lab; phương thức thứ hai ở lớp cha gọi `self.generate_markdown(**kwargs)` mà không cần biết lớp con nào đang chạy (mẫu *template method*).

```python
    def generate_markdown(self, **kwargs) -> str:
        """
        Generate the Markdown representation of the entity.
        """

        # Convert the Path object to a dictionary
        markdown = []
        markdown.append(self.generate_front_matter())

        return "\n\n".join(markdown) + "\n"
```

Mã nguồn 10: `BaseEntity.generate_markdown`: bản mặc định chỉ có frontmatter

```python
    def save_markdown(self, **kwargs) -> None:
        """
        Save the Path data to a Markdown file.
        passes kwargs to generate_markdown
        """

        write_text(self._md_path, self.generate_markdown(**kwargs))

```

Mã nguồn 11: `BaseEntity.save_markdown`: gọi `generate_markdown` của lớp con đang chạy

`model/lab.py` là lớp con nhỏ nhất, đủ để thấy toàn bộ cơ chế: `__init__` gọi `super()`, thêm hai thuộc tính riêng, rồi ghi đè `generate_markdown`.

```python
class Lab(BaseEntity):
    """
    Class representing a Lab entity.
    """

    def __init__(self,
                 id: str,
                 name: str = None,
                 description: str = None,
                 steps: dict = None):
        super().__init__(id,
                         name,
                         description
                         )
        self.steps = steps or {}

    # TODO: fetch_data() for fetching a lab's data

    def generate_markdown(self, toc_only: bool = False, **kwargs):
        """
        Generate the Markdown content for the Lab entity.
        :param toc_only: If True, only generate table of contents (structure).
        """

        markdown = []
        markdown.append(self.generate_front_matter())

        markdown.append(f"# [{self.name}]({self.url})")

        if not toc_only:
            if hasattr(self, 'description') and self.description:
                markdown.append(f"{self.description}")

        if hasattr(self, 'steps') and self.steps:
            for step_number, step_text in self.steps.items():
                markdown.append(f"## Step {step_number}: {step_text}")

        return "\n\n".join(markdown) + "\n"

```

Mã nguồn 12: `model/lab.py`: lớp con kế thừa `BaseEntity` và ghi đè `generate_markdown`

Cùng cơ chế đó ở nhánh bên phải: `Collection.fetch_catalog` là một bản duy nhất, đọc `API_URL` và `MAX_PAGES` là thuộc tính lớp; `Paths`, `Courses`, `Labs` chỉ khai lại `API_URL`. Ở v2.0.0 có ba bản copy của vòng lặp phân trang này, mỗi lớp một bản.

```python
class Courses(Collection):
    """
    Class representing a collection of courses.
    """

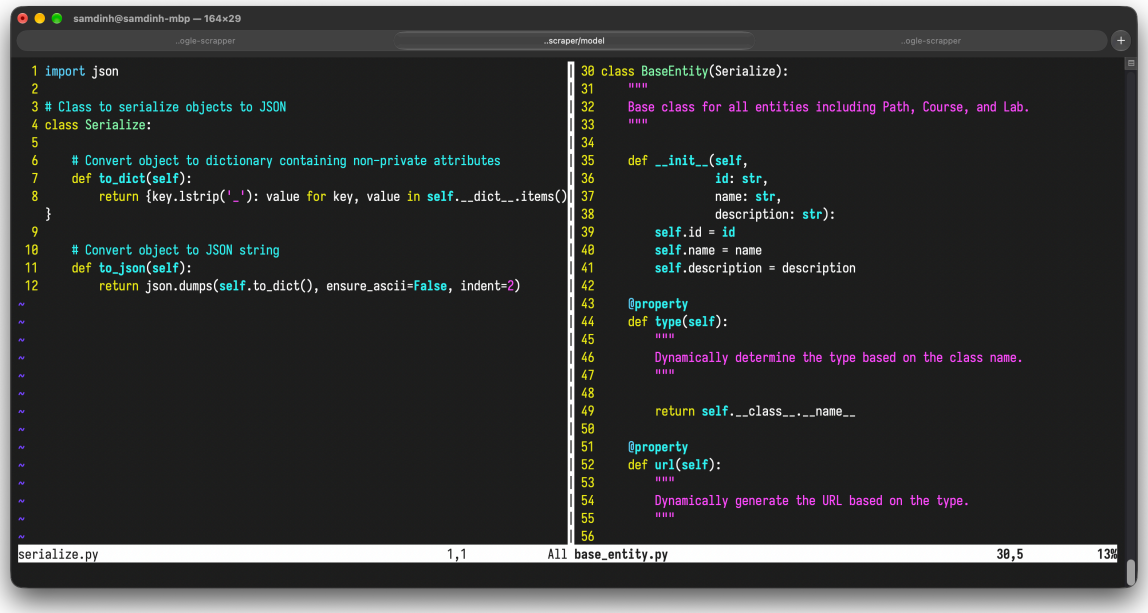
    API_URL = API_URL_COURSES

    def __init__(self,
                 name: str = None,
                 url: str = BASE_URL_COURSES,
                 collection: dict = None,
                 driver=None):
        super().__init__(name, url, collection, driver)

    def fetch_courses(self, force: bool = False) -> bool:
        """
        Gather all courses from the catalog API. Returns a Boolean to check status.
        """
        return self.fetch_catalog(force=force)

```

Mã nguồn 13: `model/courses.py`: lớp con chỉ khai `API_URL`, phần còn lại kế thừa



Hình ảnh 4: Hai cây kế thừa như IDE nhìn thấy trong mã thật

3.3.4. Đóng Gói Và Kết Hợp

3.3.4.1. Đóng Gói

- Thuộc tính "ẩn" và động: `_json_path`, `_md_path`, `_kind` là property bắt đầu bằng `_`, suy ra từ `type` và `id`.
- `to_dict` lọc khoá bắt đầu bằng `_` và khoá `driver`, nên JSON ghi ra không chứa tài nguyên hay đường dẫn máy.
    - `driver` là tài nguyên (một cửa sổ Chrome), không phải thuộc tính của thực thể, không cần lưu giữ.

- Các thuộc tính theo dõi như `scrapedTime` được thêm vào lúc ghi.

```
        the_dict = {k: v for k, v in self.__dict__.items() if not k.startswith('_')
and k != 'driver'}
        the_dict['type'] = self.type
        the_dict['url'] = self.url

        # Add scrapedTime (epoch in milliseconds)
        the_dict['scrapedTime'] = int(time.time() * 1000)

        return the_dict
```

Mã nguồn 14: `BaseEntity.to_dict`: cái gì được lưu ra đĩa

3.3.4.2. Kết Hợp (Composition)

`Course.process_lab` là chỗ một course tạo ra một lab và lưu lab đó, đồng thời cập nhật tập hợp `Labs`:

- `lab.load_json()` nạp file JSON của lab nếu đã có; đã có nghĩa là đã tải, bỏ qua để giữ tính idempotent.

```
            lab = Lab(
                id=lab_id
            )
            # Load the lab data from JSON even it's empty
            lab.load_json()

            # If the lab.name does exist, that means the lab has been extracted
already.
            if lab.name:
                print(f"(process_lab) •-• [+] Existed: {lab.id} - {lab.name}")
                return False
```

Mã nguồn 15: `Course.process_lab`: tạo `Lab`, nạp JSON, bỏ qua nếu đã có

- Nếu chưa có, tải lab về rồi `lab.save_json()` cùng `lab.save_markdown()`.

```
            # Save the lab to files.
            lab.save_json()
            lab.save_markdown()
```

Mã nguồn 16: `Course.process_lab`: lưu lab vừa tải

- Đồng thời, đưa lab vào danh sách các lab đã biết.

```
            # Add the lab to the Labs Collection
            labs_collection = Labs(name='Labs Collection')
            labs_collection.load_json()
            labs_collection.collection[lab_id] = lab.name
            labs_collection.save_json()
```

Mã nguồn 17: `Course.process_lab`: cập nhật tập hợp `Labs`

3.4. Kiểu Dữ Liệu, Hàm Và Cấu Trúc Điều Khiển

3.4.1. Kiểu Dữ Liệu

Dữ liệu của một course là `dict` lồng ba tầng, `modules` chứa `steps` chứa `activities`, mỗi tầng là `list` của `dict`; đây cũng chính là hình dạng của file JSON, nên `json.loads` và `json.dumps` đi thẳng không cần lớp chuyển đổi. Chuỗi được xử lý bằng f-string và các phương thức `str`, ví dụ `yaml_scalar` bên dưới dùng `json.dumps` để một chuỗi bất kỳ (kể cả có dấu nháy) thành một giá trị YAML hợp lệ, vì JSON là tập con của YAML. `tuple` xuất hiện làm khoá sắp xếp trong `numeric_id`: số thì so theo giá trị, chữ thì so theo chữ và đứng sau.

```
def yaml_scalar(value) -> str:
    """A string as a valid, fully escaped YAML scalar (JSON strings are YAML)."""
    return json.dumps(str(value) if value is not None else "", ensure_ascii=False)

def yaml_list(values) -> str:
    """A flow-style YAML list of strings, [] when empty."""
    return "[" + ", ".join(yaml_scalar(v) for v in values) + "]"

```

Mã nguồn 18: `yaml_scalar`, `yaml_list`: chuỗi thành YAML an toàn

```
def numeric_id(item_id) -> tuple:
    """
    Sort key for ids: numbers by value, so 9 < 10 < 108; anything else after them,
as text.
    """
    text = str(item_id)
    return (0, int(text)) if text.isdigit() else (1, text)
```

Mã nguồn 19: `numeric_id`: tuple làm khoá sắp xếp, $9 < 10 < 108$

3.4.2. Cấu Trúc Điều Khiển

Rẽ nhánh bằng `dict` (data-driven) thay cho chuỗi `if/elif`: `Course.process_step()` tra bảng `handlers` theo `activity['type']`.

- Gặp loại dữ liệu lạ (ví dụ Google mới thêm `badge`, `credential`) thì giữ nguyên dữ liệu thô, không xử lý và in thông báo.
    - Đây là chỗ thể hiện sự "trung thực" về hành vi và sự tham gia của con người: tường minh, có giới hạn, không phỏng đoán.

```
    def process_step(self, step) -> None:
        """
        Process each step in a module.
        """

        # One handler per activity kind. The set of kinds belongs to the site, not
to us:
        # a kind we have never seen (badge, credential, ...) is kept as-is and
logged.
        handlers = {
            "video": self.process_video,
            "lab": self.process_lab,
            "quiz": self.process_quiz,
            "link": self.process_link,
            "html_bundle": self.process_link,
            "document": self.process_document,
        }
```

Mã nguồn 20: `Course.process_step`: bảng dispatch theo loại activity

```

        for activity in step['activities']:
            # Fix HREF if it is a session URL
            if activity.get('href') and '/course_sessions/' in activity['href']:
                 activity['href'] = re.sub(r'/course_sessions/[^/]+', f'/
course_templates/{self.id}', activity['href'])

            activity_type = activity['type']
            activity_full_url = f"{BASE_URL}{activity['href']}"

            handler = handlers.get(activity_type)
            if handler:
                handler(activity, activity_full_url)
            else:
                print(f"(process_step) •-> {activity_type}: {activity['id']:>6} -
{activity['title']} (kept as-is, no handler)")
```

Mã nguồn 21: `Course.process_step`: loại lạ được giữ nguyên và báo

3.4.3. Vòng Lặp Và Hàm

Đệ quy, vì `items` của một bài học có thể lồng nhau: `Course._parse_lesson_item()` xử lý một item rồi gọi chính nó cho các item con.

```python
    def _parse_lesson_item(self, item: dict) -> str:
        """
        Parse a single item from the lesson data into Markdown/HTML.
        """
        # Based on observed structure, items have 'heading', 'paragraph', 'list',
etc.
        # This is a best-effort parser.

        # Check for nested items (some structures might be recursive or wrapped)
        if 'items' in item:
             sub_content = []
             for sub in item['items']:
                 parsed = self._parse_lesson_item(sub)
                 if parsed:
                     sub_content.append(parsed)
             return "\n\n".join(sub_content)
```

Mã nguồn 22: `Course._parse_lesson_item`: đệ quy trên cây item

Lặp có ngắt: `Collection.fetch_catalog()` phân trang bằng `while True` với một "cầu chì" `MAX_PAGES`, dừng khi trang trả về rỗng hoặc vượt giới hạn; dùng khi lấy danh sách toàn bộ Path, Course, Lab có trên skills.google.

```python
    def fetch_catalog(self, force: bool = False) -> bool:
        """
        Gather every item of this collection from the paged catalog API,
        read through the signed-in browser (the JSON lands in a <pre> tag).
        Returns True on success.
```

Mã nguồn 23: `Collection.fetch_catalog`: chữ ký và tham số mặc định

```python

                for item in items:
                    title = item.get("title")
                    item_path = item.get("path")
                    if title and item_path:
                        # /paths/16?locale=en -> 16 : drop the query, keep the last
segment
                        item_id = item_path.split('?')[0].split('/')[-1]
                        if item_id:
                            items_found[item_id] = title.strip()

                page += 1
                if page > self.MAX_PAGES:
                    print(f"\nReached safety limit of {self.MAX_PAGES} pages.")
                    break

            print(f"\nTotal {label} found: {len(items_found)}")
```

Mã nguồn 24: `Collection.fetch_catalog`: vòng lặp phân trang có cầu chì

Về hàm: tham số có giá trị mặc định (`force=False`, `toc_only=False`), `**kwargs` chuyển tiếp từ `save_markdown` sang `generate_markdown`, hàm trả `bool` để người gọi quyết định đi tiếp hay dừng (`extract_course_metadata`, `ensure_authenticated`), và `@staticmethod` cho hàm không cần `self` (`Path.menu_activities`).

3.5. Xử Lý Ngoại Lệ

> **Nguyên Tắc**
>
> Một *activity* hỏng không làm hỏng cả *course* trong quá trình `fetch`; một ngắt giữa chừng không để lại nửa file; mọi lỗi đều được báo, không có lỗi im lặng.

Các ngoại lệ gặp trong mã thuộc ba nhóm:

- Của thư viện: `NoSuchElementException` (Selenium) khi trang thiếu phần tử mong đợi, `json.JSONDecodeError` khi danh mục không phải JSON, `OSError` khi ghi file.
- Của Python: `KeyboardInterrupt` và `EOFError` khi người dùng dừng ở bước đăng nhập, `KeyError` khi tra bảng trình duyệt, `ValueError` khi `type` lạ.
- Bắt rộng có chủ ý: mỗi `process_*` của một activity nằm trong `try/except Exception` riêng và in lỗi kèm tên phương thức, để course đi tiếp sang activity kế. Bản IE221 không định nghĩa lớp ngoại lệ riêng; phần thử lại khi mất mạng (P10 ở Phụ Lục A.3) là hướng phát triển.

`services/store.py` ghi file nguyên tử (atomic): ghi vào file tạm cùng thư mục, `fsync`, rồi `os.replace`; nếu có lỗi hoặc Ctrl+C giữa chừng thì xoá file tạm và ném lại ngoại lệ, không bao giờ để lại "nửa" file.

```python
def write_text(path, text: str) -> None:
    """
    Write UTF-8 text atomically: tmp file in the same folder, fsync, then
    os.replace(). A Ctrl+C can never leave half a file behind.
    """
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    fd, tmp = tempfile.mkstemp(dir=path.parent, prefix=".tmp-")
    try:
        with os.fdopen(fd, "w", encoding="utf-8", newline="\n") as textfile:
            textfile.write(text)
            textfile.flush()
            os.fsync(textfile.fileno())
        os.replace(tmp, path)
    except BaseException:
        try:
            os.remove(tmp)
        except OSError:
            pass
        raise
```

Mã nguồn 25: `store.write_text`: ghi nguyên tử với `try/except/raise`

`services/browser.py` xử lý trường hợp người dùng bỏ dở thao tác đăng nhập: `input()` bị ngắt bằng Ctrl+C hay Ctrl+D được bắt và trả `False`, người gọi dừng tường minh.

```
    while "sign_in" in driver.current_url:
        suffix = f" for {what}" if what else ""
        print(f"\n\033[93m[!] Authentication required{suffix}.\033[0m")
        print("Please sign in to the browser window if you haven't.")
        try:
            input("Press Enter after you have signed in and the page is loaded...
")
        except (KeyboardInterrupt, EOFError):
            print("\n\033[91mAborted authentication.\033[0m")
            return False
        driver.get(url)

        if "sign_in" in driver.current_url:
            print("\n\033[91m[!] Still on the sign-in page. Finish signing in
before pressing Enter.\033[0m")
    return True
```

Mã nguồn 26: `browser.ensure_authenticated`: bắt `KeyboardInterrupt`, `EOFError`

`BaseEntity.save_json` bắt `OSError` của bước ghi và chỉ cập nhật chỉ mục khi ghi xong, nên chỉ mục không bao giờ trỏ tới một file không tồn tại.

```
    def save_json(self):
        """
        Write the entity to its JSON file (atomically) and update the index.
        """
        entity_data = self.to_dict()
        try:
            write_json(self._json_path, entity_data)
        except OSError as e:
            print(f"(BaseEntity.save_json) Error writing JSON to file:
{self._json_path}")
            print(e)
            return

        index = Index()
        index.upsert(self._kind, entity_data)
        index.save()
```

Mã nguồn 27: `BaseEntity.save_json`: ghi trước, cập nhật chỉ mục sau

3.6. Phiên Đăng Nhập Và Thu Thập

Từ đầu 2026, trang course trên skills.google chuyển hướng về trang đăng nhập nếu không có phiên; vì vậy mọi trang đều đi qua một trình duyệt Chrome (hoặc Edge) do Selenium *[6]* điều khiển, `requests` chỉ còn dùng để tải tài liệu PDF với cookie lấy từ trình duyệt. Toàn bộ việc này nằm trong `services/browser.py`, khoảng một trăm dòng, với ba hàm công khai:

- `launch_browser` mở trình duyệt trên một *profile* cố định (`.webdriver_profiles` theo mặc định), nên phiên đăng nhập được giữ qua các lần chạy: đăng nhập một lần, những lần sau đi thẳng. Một danh sách cờ dùng chung cho Chrome và Edge; `--headless=new` khi profile đã đăng nhập.

- `ensure_authenticated` là **chỗ duy nhất** chờ người dùng: nếu URL hiện tại chứa `sign_in` thì in hướng dẫn, chờ Enter, tải lại trang, lặp tới khi qua được hoặc người dùng bỏ. Ở v2.0.0 có sáu khối `input()` rải trong `course.py`.
- `get_page` gói hai bước trên và trả về cây `BeautifulSoup` *[7]* của trang đã render; `model/` chỉ gọi hàm này, không biết Selenium.

```
    try:
        options_class, service_class, driver_class = BROWSERS[browser.lower()]
    except KeyError:
        raise ValueError(f"Unsupported browser: {browser} (chrome or edge)")

    options = options_class()
    for argument in BROWSER_ARGUMENTS:
        options.add_argument(argument)
    options.add_experimental_option("excludeSwitches", ["enable-logging", "enable-
automation"])

    # The "new" headless mode is the one that behaves like a real window.
    if headless:
        options.add_argument("--headless=new")

    if profile_folder:
        options.add_argument(f"user-data-dir={os.path.abspath(profile_folder)}")

    return driver_class(service=service_class(), options=options)
```

Mã nguồn 28: `browser.launch_browser`: tra bảng trình duyệt, cờ dùng chung, profile cố định

```
def open_page(driver, url: str, what: str = "") -> bool:
    """
    Navigate the browser to url, dealing with the sign-in redirect.
    Returns False when there is no browser or the user aborted the sign-in.
    """
    if not driver:
        print(f"(open_page) A signed-in browser is required{' for ' + what if what
else ''}.")
        return False
    driver.get(url)
    return ensure_authenticated(driver, url, what)

def get_page(driver, url: str, what: str = "") -> BeautifulSoup | None:
    """
    open_page(), then hand back the rendered page as BeautifulSoup.
    """
    if not open_page(driver, url, what):
        return None
    return BeautifulSoup(driver.page_source, "html.parser")
```

Mã nguồn 29: `browser.open_page`, `browser.get_page`: cửa duy nhất tới trang web cho `model/`

Việc thu thập một course (`Course.extract_transcript`) đi theo bốn bước, mỗi bước là một phương thức:

1. `extract_course_metadata` đọc khối `ld+json` của trang: `@id`, `name`, `datePublished`, `about` (topics), `teaches` (objectives); mô tả lấy từ `meta[name=description]`.
2. `extract_course_outline` đọc đề cương từ thẻ `ql-contents-menu`, cho ra cây `modules` → `steps` → `activities`.
3. `process_step` cho từng step, dispatch theo loại activity (Mã nguồn 20): video thì lấy transcript YouTube, lab thì tạo `Lab`, quiz thì lấy câu hỏi, document thì tải file, link và html_bundle thì giữ URL.
4. `save_json` rồi `save_markdown`.

Quy tắc bỏ qua nằm ở bước đầu: nếu `datePublished` trên trang bằng giá trị đã lưu trong JSON và không có `--force`, phương thức in lý do và trả `False`, course dừng tại đó; đây là FR-05 và cũng là lý do chạy lại là idempotent.

```
            course_objectives_json = json.loads(course_ld_json_text)
            datePublished = course_objectives_json.get('datePublished')

            # If the course has the same datePublished, return False and not
continue.
            if not force and datePublished == self.datePublished:
                print(f"(extract_course_metadata) Course {self.id} already
extracted. datePublished: {datePublished}\n")
                return False

            self.id = course_objectives_json.get('@id').split('/')[-1]
            self.name = course_objectives_json.get('name').strip()
            self.description = course_description
            self.datePublished = course_objectives_json.get('datePublished')
            self.topics = course_objectives_json.get('about')
            self.objectives = course_objectives_json.get('teaches')
```

Mã nguồn 30: `Course.extract_course_metadata`: bỏ qua theo `datePublished`, tường minh

Với path, `Path.fetch_data` đọc `ql-contents-menu[modules]` của trang path làm nguồn sự thật về thành viên, vì `href` ở đó phân biệt `/course_templates/<id>` với `/focuses/<id>` (lab), còn `ld+json.hasPart` chỉ dùng để bổ sung tên và mô tả; `fetch -p ID --cascade` sau đó lần lượt thu thập từng course trong path. Danh mục (`list -r`) cũng đi qua trình duyệt: endpoint `/catalog/list?format[]=...` trả 403 cho `requests` trần, nhưng khi Chrome mở nó thì JSON nằm trong thẻ `<pre>`, `fetch_catalog` đọc thẻ đó rồi `json.loads`.

3.7. Lưu Trữ

Không có cơ sở dữ liệu. Mỗi thực thể là một file JSON và file đó là nguồn sự thật (SSOT); mọi thứ khác đều dẫn xuất và dựng lại được.

```
data/
├── index.json          chỉ mục dẫn xuất: {kind: {id: {name,
datePublished, scrapedTime}}}
├── paths.json          tập hợp {id: name} của Paths (Collection)
├── paths/280.json      một path
├── courses/892.json    một course, đầy đủ modules/steps/activities
└── labs/<id>.json      một lab
```

Mã nguồn 31: Thư mục `data/`: một thực thể, một file

`services/store.py` cung cấp đúng những gì `model/` cần:

- `write_text`, `write_json`, `read_json`: đọc ghi UTF-8, ghi luôn nguyên tử (Mã nguồn 25).
- `Index`: lớp bọc `data/index.json`. `upsert` trộn ba trường chỉ mục của một document vào bảng và giữ giá trị cũ cho trường document không mang (một dòng danh mục trần chưa có ngày); `search` tìm chuỗi con không phân biệt hoa thường, trả lời từ chỉ mục khi hỏi `name` / `id`, mở file thực thể khi hỏi trường khác; `rebuild` quét `data/<kind>/*.json` và dựng lại toàn bộ, đó là lệnh `reindex`.

```python
    def upsert(self, kind: str, doc: dict) -> None:
        """
        Merge the index fields of doc into the table, keeping older values
        for fields the doc does not carry (a bare catalog entry has no dates).
        """
        entity_id = str(doc.get("id", "")).strip()
        if not entity_id:
            return
        entry = self.tables.setdefault(kind, {}).setdefault(entity_id, {})
        for field in INDEX_FIELDS:
            if doc.get(field) not in (None, ""):
                entry[field] = doc[field]

    def rebuild(self) -> dict:
        """
        Rebuild every table from data/<kind>/*.json. Returns counts per kind.
        """
        self.tables = {kind: {} for kind in KINDS}
        for kind in KINDS:
            for entity_file in sorted((self.data_dir / kind).glob("*.json")):
                if entity_file.name.endswith("-prompt.json"):
                    continue
                doc = read_json(entity_file)
                if isinstance(doc, dict):
                    doc.setdefault("id", entity_file.stem)
                    self.upsert(kind, doc)
        self.save()
        return {kind: len(table) for kind, table in self.tables.items()}

```

Mã nguồn 32: `Index.upsert` và `Index.rebuild`: chỉ mục luôn dựng lại được từ file

`BaseEntity.load_json` chỉ là `self.__dict__.update(data)`: đối tượng Python và file JSON có cùng hình dạng (Mã nguồn 27), nên không cần tầng ORM hay schema riêng. Ở v2.0.0, TinyDB (`database.json`) chạy song song với các file JSON; bản này bỏ TinyDB

vì hai nguồn sự thật phải được giữ đồng bộ mà không đem lại gì hơn một `index.json` dựng lại được trong một giây.

3.8. Kết Xuất Markdown

Lệnh `md` (và bước cuối của `fetch`) biến JSON thành note Markdown cho Obsidian, hoàn toàn offline. Mỗi note gồm frontmatter YAML và thân bài.

Frontmatter do `BaseEntity.generate_front_matter` sinh, là tập mở rộng (superset) của OKF v0.2 *[3]*: các khoá OKF (`type`, `title`, `description`, `resource`, `tags`, `sources`, `generated`), hai khoá riêng của công cụ (`id`, `portal`), và các tên cũ mà vault hiện có đang truy vấn (`url`, `date_published`, `topics`, `scraped_date`). Mọi giá trị vô hướng đi qua `yaml_scalar` (Mã nguồn 18), nên tên course có dấu nháy đơn không còn làm hỏng YAML như ở v2.0.0.

```
        lines = ["---"]
        lines.append(f"id: {yaml_scalar(self.id)}")
        lines.append(f"title: {yaml_scalar(self.name)}")
        lines.append(f"type: {self.type}")
        lines.append("portal: public")
        if description:
            lines.append(f"description: {yaml_scalar(first_sentence(description))}")
        lines.append(f"resource: {self.url}")
        if topics:
            lines.append(f"tags: {yaml_list(topics)}")
        lines.append("sources:")
        lines.append(f"  - resource: {self.url}")
        lines.append(f"    title: {yaml_scalar(f'Google Skills {self.type.lower()}
{self.id}')}")
        lines.append("    author: Google Cloud")
        if date_published:
            lines.append(f"    last_modified: {date_published}")
        lines.append("generated:")
        lines.append(f"  by: skills-scraper/{__version__}")
        lines.append(f"  at: {scraped_at.isoformat(timespec='seconds')}")
        # Legacy names, kept so existing vault queries keep working.
        lines.append(f"url: {self.url}")
        if date_published:
            lines.append(f"date_published: {date_published}")
        if hasattr(self, 'topics'):
            lines.append(f"topics: {yaml_list(topics)}")
        lines.append(f"scraped_date: {scraped_at.strftime('%Y-%m-%d')}")
        lines.append("---")
        return "\n".join(lines)
```

Mã nguồn 33: `BaseEntity.generate_front_matter`: khoá OKF, khoá riêng và alias cũ

Kết quả thật cho course 892, sinh bằng `uv run skills-scraper md -c 892 --toc` từ `docs/samples/data/` trong repo, không cần mạng:

```yaml
---
id: "892"
title: "Conversational AI on Vertex AI and Dialogflow CX"
type: Course
portal: public
description: "In this course you will learn how to use the new
generative AI features in Dialogflow CX ..."
resource: https://www.skills.google/course_templates/892
sources:
  - resource: https://www.skills.google/course_templates/892
    title: "Google Skills course 892"
    author: Google Cloud
    last_modified: 2023-11-21
generated:
  by: skills-scraper/2.0.1
  at: 2026-10-16T14:16:02+00:00
url: https://www.skills.google/course_templates/892
date_published: 2023-11-21
topics: []
scraped_date: 2026-10-16
---

# [Conversational AI on Vertex AI and Dialogflow CX](https://www.
skills.google/course_templates/892)

## Course Introduction

### Video - [Course Introduction](https://www.skills.google/course_
templates/892/video/425551)

- [YouTube: Course Introduction](https://www.youtube.com/watch?v=
JiLHQnBW92U)
```

Mã nguồn 34: Note `courses/Conversational-AI-on-Vertex-AI-and-Dialogflow-CX.md`, rút gọn

Quy ước thân bài: tiêu đề cấp 1 là tên course có liên kết về trang gốc; mỗi module một tiêu đề cấp 2; mỗi activity một tiêu đề cấp 3 có loại và liên kết; video kèm liên kết YouTube và transcript (bỏ khi `--toc` hoặc `--no-transcript`); quiz là callout với từng câu hỏi; tài liệu PDF được tải vào `<vault>/materials/courses/<id>/` và liên kết tương đối từ note. Note của path liệt kê các course và lab của nó bằng liên kết tương đối `../courses/...`, `../labs/...`; chính các liên kết này tạo nên đồ thị ở Hình ảnh 1 mà không ai thiết kế trước.

3.9. Hai Runner

Cả hai runner đều chỉ `import` từ `model/` và `services/`; không có logic nghiệp vụ nào trong `cli.py` hay `tui.py`. Thêm một runner thứ ba (chẳng hạn một HTTP API) sẽ không chạm tới `model/`.

3.9.1. Giao Diện Dòng Lệnh

`cli.py` dùng `argparse` với một sub-parser cho mỗi lệnh, mỗi lệnh gắn một hàm `cmd_*` qua `set_defaults(func=...)`; `main()` chỉ phân tích tham số rồi gọi `args.func(args)`.

Lệnh	Làm gì	Cờ chính
`list (l)`	Liệt kê path, course hoặc lab từ `data/`	`-p` / `-c` / `-l`, `-r` làm mới từ site, `-i` sắp theo id, `--headless`
`fetch (f)`	Thu thập vào `data/` và sinh Markdown	`-p ID...` / `-c ID...` / `-l ID...`, `--cascade`, `--force`, `--toc`, `--no-transcript`, `--no-md`
`md`	Sinh Markdown từ JSON đã có, offline	`-c ID,ID` / `-p ID` / `-l ID`, `--toc`, `--no-transcript`
`search (s)`	Tìm trong dữ liệu đã tải	`QUERY`, `-c` / `-p` / `-l`, `--field name|topics|...`
`reindex`	Dựng lại `data/index.json` từ các file thực thể	—
`browser (b)`	Mở trình duyệt đã đăng nhập và chờ, để đăng nhập	`--profile-folder`

Bảng 7: Các lệnh của `skills-scraper`, theo `--help` thật

```python
def main():
    parser = argparse.ArgumentParser(description="CloudSkillsBoost Scraper CLI")
    subparsers = parser.add_subparsers(dest='command', help='Command to execute')

    # List command
    parser_l = subparsers.add_parser('list', aliases=['l'], help='List all paths,
courses, or labs')

    # Mutually exclusive group for type
    group_type = parser_l.add_mutually_exclusive_group()
    group_type.add_argument('--paths', '-p', action='store_true', help='List all
paths (default)')
    group_type.add_argument('--courses', '-c', action='store_true', help='List all
courses')
    group_type.add_argument('--labs', '-l', action='store_true', help='List all
labs')

    # Reload flag
    parser_l.add_argument('--reload', '-r', action='store_true', help='Reload list
from remote before listing')
    parser_l.add_argument('--headless', action='store_true', help='Run the browser
without a window (profile must be signed in)')

    # Mutually exclusive group for sorting
    group_sort = parser_l.add_mutually_exclusive_group()
    group_sort.add_argument('--name', '-n', action='store_true', help='Sort by name
(default)')
    group_sort.add_argument('--id', '-i', action='store_true', help='Sort by ID')

    parser_l.set_defaults(func=cmd_list)
```

Mã nguồn 35: `cli.main`: sub-parser `list` với hai nhóm loại trừ lẫn nhau

```python
def cmd_reindex(args):
    """Rebuild data/index.json from the entity files"""
    counts = Index().rebuild()
    for kind, count in counts.items():
        print(f"{kind:>8}: {count}")

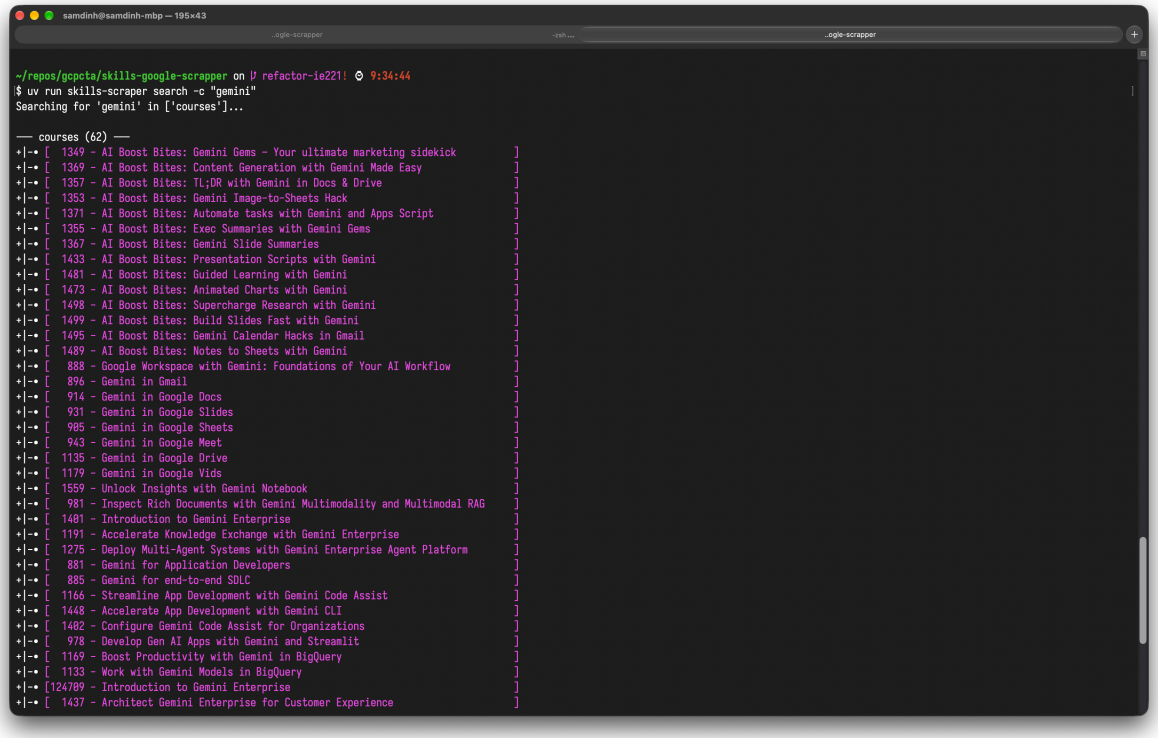
```

Mã nguồn 36: `cli.cmd_reindex`: một lệnh CLI là một hàm gọi vào `services`

Vì giao tiếp bằng file và mã thoát, CLI ghép được với shell và các công cụ khác mà không cần API riêng, ví dụ thu thập mọi course của một path rồi xem site đã đổi gì so với lần trước:

```shell
uv run skills-scraper fetch -p 280 --cascade --toc
jq -r '.courses | keys[]' data/paths/280.json | while read id; do
  uv run skills-scraper md -c "$id" --toc
done
git -C csbmdvault diff --stat
```

Mã nguồn 37: Ghép CLI với `jq`, `while` của shell và `git`



Hình ảnh 5: `skills-scraper` chạy trong terminal

3.9.2. Giao Diện Tương Tác

`tui.py` giữ nguyên kiểu menu của v2.0.0: menu số và chữ, màu ANSI, vòng lặp `while` đọc `input()` và rẽ nhánh theo lựa chọn (`c` một course, `p` một path, `l` danh sách, `h` tất cả course, `w` mở trình duyệt, `q` thoát). Điểm mới duy nhất là trình duyệt dùng chung: `CloudSkillsBoost.driver` là một `@property` khởi tạo lười (lazy), chỉ mở Chrome khi

lựa chọn đầu tiên cần tới mạng, và `close()` đóng nó trong khối `finally` của `main()`, nên thoát bằng `q` hay Ctrl+C đều không để lại cửa sổ Chrome mồ côi.

```
    @property
    def driver(self):
        """The signed-in browser, launched on first use and reused after that."""
        if self._driver is None:
            print("\n\033[35mLaunching the browser...\033[0m\n")
            self._driver = launch_browser()
        return self._driver

    def close(self):
        """Quit the browser if one was launched."""
        if self._driver is not None:
            self._driver.quit()
            self._driver = None

```

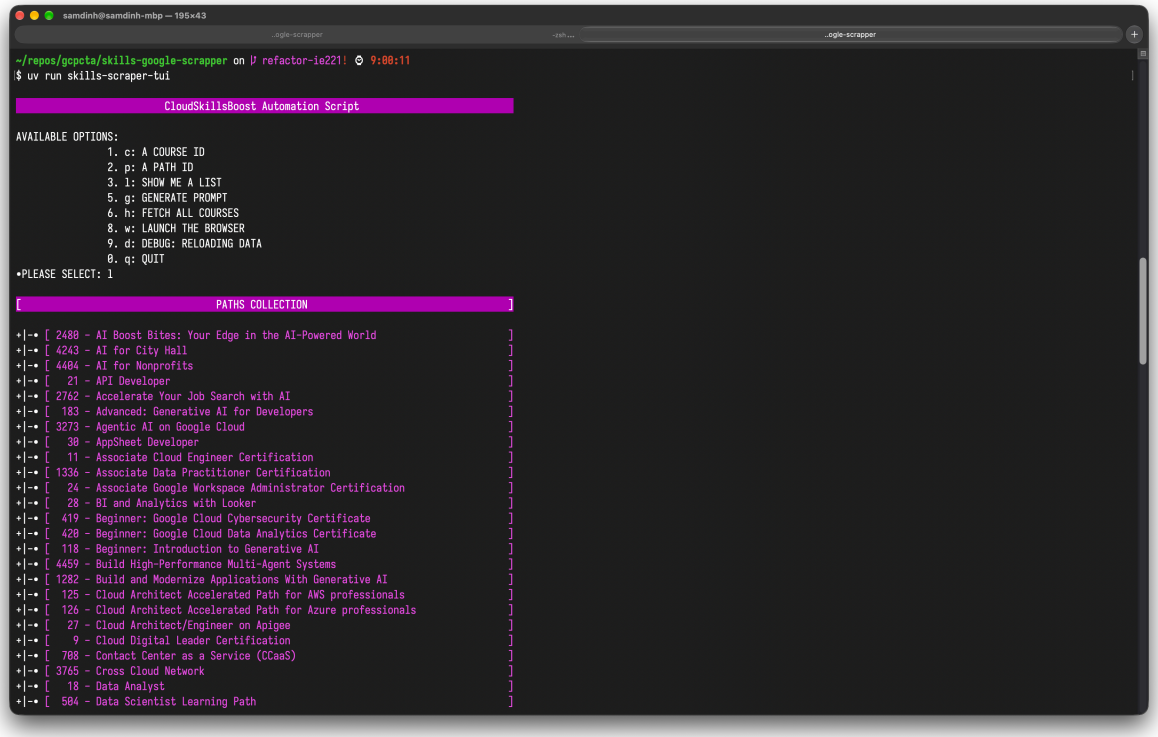
Mã nguồn 38: `CloudSkillsBoost.driver`: trình duyệt khởi tạo lười, dùng chung cho cả phiên

```
    print("\033[45m"
          f"{app_title:^87}"
          "\033[0m")
    print()

    # Create an instance of CloudSkillsBoost and start interactive mode
    cloud_skills_boost = CloudSkillsBoost()
    try:
        cloud_skills_boost.interactive_mode()
    finally:
        cloud_skills_boost.close()

    sys.exit(0)
```

Mã nguồn 39: `tui.main`: `try/finally` đảm bảo đóng trình duyệt khi thoát



Hình ảnh 6: `skills-scraper-tui` : cùng lõi, giao diện khác

3.10. Kết Chương

Chương này chỉ ra trên mã thật của branch `refactor-ie221` các nội dung của môn học: gói và module với `uv` và `src/` (Mã nguồn 6); hai cây kế thừa từ `Serialize` với `@property` , `super()` và ghi đè (Hình ảnh 3); kiểu dữ liệu lồng, dispatch bằng `dict` , đệ quy và vòng lặp có cầu chì; ngoại lệ được bắt ở đúng phạm vi cùng ghi file nguyên tử (Mã nguồn 25). Ba mảng còn lại, phiên đăng nhập đi qua một cửa duy nhất (Mã nguồn 29), lưu trữ bằng file JSON làm nguồn sự thật (Mã nguồn 31) và kết xuất Markdown theo OKF (Mã nguồn 33), tạo nên công cụ hoàn chỉnh mà hai runner (Bảng 7, Hình ảnh 6) chỉ là hai lớp vỏ mỏng. Chương 4 trả lời câu hỏi: làm sao biết tất cả những điều trên vẫn đúng khi site đổi.

CHƯƠNG 4.

KIỂM THỬ, VẬN HÀNH VÀ BẢO TRÌ

Một scraper sống bằng cách bám theo một website không thuộc về mình, nên câu hỏi quan trọng hơn "có chạy không" là "làm sao biết nó còn chạy đúng, và khi không còn đúng thì biết ở đâu". Chương này trả lời bằng bốn mảng: bộ test offline trên các trang đã ghi lại, tính idempotent khi chạy lại, quy trình khi nền tảng thay đổi với các thay đổi thật từ v2.0.0 tới nay, và cấu hình ba lớp cho tính di động.

4.1. Kiểm Thử Offline

Mọi test chạy không cần mạng và không cần Chrome. Nguồn vào là các trang đã ghi lại từ site trong `tests/fixtures/google_skills/`: trang course 72 (`course-72.html`), riêng thẻ `ql-contents-menu` của nó, cây `modules` đã phân tích (`course-72-modules.json`, có mang loại `badge` mà công cụ không có handler), và trang video 613054 trước và sau khi render. Test khẳng định trên **cấu trúc** (có `datePublished`, mỗi activity có `id`, `type`, `title`, `href`), không trên **giá trị** (tiêu đề course 72 đã đổi trong bảy tháng), nên fixture không phải ghi lại mỗi khi Google sửa chữ.

Tệp	Kiểm tra	Số test
`test_course.py`	Metadata từ `ld+json`; bỏ qua khi `datePublished` không đổi; đề cương từ `ql-contents-menu`; Markdown render được mọi loại activity kể cả loại lạ; danh sách dùng dấu gạch	5
`test_frontmatter.py`	Dấu nháy trong tiêu đề vẫn là YAML hợp lệ; đủ khoá OKF và alias cũ; mọi loại thực thể đều có `type`	3
`test_path.py`	`ql-contents-menu` phân biệt lab với course; không có menu thì lùi về `ld+json`; note path liên kết lab tới `labs/`	3
`test_store.py`	`write_json` không để lại file tạm; `upsert` giữ trường cũ; `search` theo tên rồi theo nội dung file; `rebuild` bỏ qua file prompt	4

Bảng 8: Bộ test offline, 15 test trong 4 tệp

Kết quả chạy từ một bản clone sạch trên máy ảo Linux (aarch64), Python 3.12.14 do `uv` tải, ngày 16/10/2026:

```
$ uv sync
Using CPython 3.12.14
Creating virtual environment at: .venv
Resolved 30 packages in 0.56ms
Installed 27 packages in 3ms
$ uv run pytest -q
...............
[100%]
15 passed in 0.20s
```

Mã nguồn 40: `uv sync` và `uv run pytest` từ clone sạch: 15/15, không cần mạng

Bộ test còn đóng vai trò **cảnh báo sớm**: khi Google đổi markup, bước đầu tiên của người bảo trì là ghi lại trang mới vào `tests/fixtures/`; test nào đỏ chỉ đúng chỗ cần sửa, trước khi chạy bất kỳ lệnh nào lên site thật.

4.2. Chạy Lại Và Tính Idempotent

NFR-01 yêu cầu chạy lại cùng một lệnh không làm hỏng và không làm thừa. Ba cơ chế trong mã bảo đảm điều đó, mỗi cơ chế đều đã được chỉ ra ở chương 3:

- `fetch -c 892` lần hai: `extract_course_metadata` so `datePublished` trên trang với giá trị đã lưu, bằng nhau thì in `Course 892 already extracted. datePublished: 2023-11-21` và trả `False`, course dừng tại đó (Mã nguồn 30); `--force` để ép thu thập lại. Hành vi này có test (`test_metadata_skips_when_date_published_unchanged`).
- Lab đã có file JSON thì `process_lab` không tải lại (Mã nguồn 15), vì một lab xuất hiện trong nhiều course và nhiều path.
- Ctrl+C giữa chừng: mọi file đi qua `write_text` nguyên tử (Mã nguồn 25), nên trên đĩa hoặc là bản cũ nguyên vẹn hoặc là bản mới hoàn chỉnh, không bao giờ là nửa file; `test_write_json_leaves_no_temp_files` kiểm tra không còn `.tmp-*` sau khi ghi.

Lệnh `md` thì thuần hàm: cùng JSON vào, cùng Markdown ra; chạy bao nhiêu lần cũng được và `git diff` trên vault rỗng nếu dữ liệu không đổi. Đây cũng là cách phát hiện site đã đổi gì: `fetch --force` rồi `git diff` vault.

Đường đi qua Chrome (`fetch`, `list -r`, `--cascade`) đã được sinh viên chạy thật trên macOS ngày 26/09/2026 với các lệnh `uv run skills-scraper list -p -r`, `fetch -c 892 --toc`, `fetch -p 280 --cascade --toc`; ba giả định lấy từ dòng chính (JSON danh mục nằm trong `<pre>`, `driver.current_url` chứa `sign_in` khi bị chuyển hướng, `href` dạng `/focuses/<id>` cho lab trong menu của path) đều đúng với site hiện tại.

4.3. Khi Nền Tảng Thay Đổi

Quy trình xử lý thay đổi của site gồm ba bước, như đã nêu ở Phần 2.2: ứng dụng phát hiện và báo (log, test thất bại, dừng rõ ràng); con người đánh giá giá trị và chi phí; thay đổi được thực hiện thành một commit có nguồn và lý do. Bảng dưới là các thay đổi thật của skills.google từ v2.0.0 (02/2026) tới nay, cách dòng chính (Go) đã xử lý, và commit tương ứng trên `refactor-ie221` đưa cách xử lý đó về lại Python.

Thay đổi của site	Triệu chứng ở v2.0.0	Cách xử lý	Dòng chính	refactor-ie221
Trang course bắt buộc đăng nhập (07/2026)	`requests.get` nhận trang sign-in, hoặc dừng chờ ở sáu chỗ	mọi trang qua trình duyệt, `ensure_authenticated` một chỗ	`7637ba2`	`9cc37c4`
API danh mục trả 403 cho request trần	`list -r` rỗng	đọc JSON trong `<pre>` qua trình duyệt, một `fetch_catalog` cho ba lớp	`61f2cbf`	`ae21659`
Loại activity mới: `html_bundle`, `badge`, `credential`	bị bỏ qua lặng lẽ	dispatch bằng `dict`, loại lạ giữ nguyên và log	`31390c6`	`9cfcd03`
`ld+json.hasPart` gắn nhãn lab thành Course	path trỏ sai course	`ql-contents-menu` là nguồn sự thật, `ld+json` chỉ bổ sung	`0b3c374`	`1167943`
Tên course có dấu nháy đơn	frontmatter YAML hỏng, Obsidian báo lỗi	`json.dumps` cho giá trị vô hướng, `title` thay `name`	`1aa2f3e`	`23d5a63`

Bảng 9: Site đổi, mã đổi theo: năm thay đổi thật và commit xử lý

Mỗi dòng của bảng là một lần con người quyết định "đáng để theo"; không có dòng nào do công cụ tự sửa. Những gì chưa theo (ví dụ partner portal, phần Lab chạy thật) cũng là quyết định có lý do và được ghi ở Phần 5.2.

4.4. Cấu Hình Và Tính Di Động

NFR-03 và NFR-06 được đáp ứng bằng `config.py` với ba lớp cấu hình, lớp sau ghi đè lớp trước:

1. Mặc định trong mã (`DEFAULTS`): `data/`, `csbmdvault/`, `.webdriver_profiles/`, Chrome, không headless; đủ để chạy ngay sau `uv sync`.
2. `config.yaml` (dữ liệu, không phải mã): tìm theo `$CSB_CONFIG`, rồi `./config.yaml`, rồi `./config/config.yaml`; đường dẫn tương đối tính từ thư mục chứa file, nên `vault: ../skills-vault` nghĩa là một chỗ bất kể chạy lệnh từ đâu. Repo kèm `config.example.yaml` có chú thích từng khoá.

3. Biến môi trường `CSB_DATA`, `CSB_VAULT`, `CSB_PROFILE_DIR`, `CSB_BROWSER`, `CSB_HEADLESS`: ghi đè từng khoá, tiện cho script và CI; chuỗi `"1"`, `"true"`, `"yes"`, `"on"` được hiểu là bật.

```
    settings = {section: dict(values) for section, values in DEFAULTS.items()}
    base_dir = Path.cwd()

    config_file = find_config_file()
    if config_file:
        base_dir = config_file.parent
        with open(config_file, encoding="utf-8") as yamlfile:
            user = yaml.safe_load(yamlfile) or {}
        for section, values in user.items():
            if section in settings and isinstance(values, dict):
                settings[section].update(values)

    for (section, key), env_name in ENV_KEYS.items():
        if env_name in os.environ:
            settings[section][key] = os.environ[env_name]

    for key, value in settings["paths"].items():
        settings["paths"][key] = (base_dir / str(value)).resolve()

    headless = settings["browser"]["headless"]
    if isinstance(headless, str):
        settings["browser"]["headless"] = headless.strip().lower() in ("1", "true",
"yes", "on")

    return settings
```

Mã nguồn 41: `config.load_settings`: trộn ba lớp, đường dẫn tương đối tính từ `config.yaml`

Ở v2.0.0, `settings.py` là mã Python chứa đường dẫn tuyệt đối của máy tác giả; bản này không còn chuỗi `/Users/` nào trong `src/` (kiểm bằng `grep -rn "/Users/" src`, kết quả rỗng), nên repo chạy được trên máy khác, hệ điều hành khác mà không sửa mã. Chính nhờ vậy toàn bộ phần offline của chương này (Mã nguồn 40, Mã nguồn 34) được kiểm trên một máy ảo Linux chỉ bằng ba biến môi trường trỏ vào `docs/samples/data`, trong khi phần cần Chrome chạy trên macOS của sinh viên.

4.5. Kết Chương

Công cụ được kiểm theo hai lớp: 15 test offline trên trang đã ghi lại (Bảng 8), chạy xanh từ clone sạch trong một phần tư giây (Mã nguồn 40), và một lần chạy thật qua Chrome trên macOS cho các đường cần đăng nhập. Tính idempotent đến từ ba cơ chế có trong mã và có test; việc bám theo site là một quy trình có con người ở giữa, với năm thay đổi thật đã được xử lý và ghi thành commit (Bảng 9); cấu hình ba lớp (Mã nguồn 41) làm cho repo di động và cũng là cái cho phép kiểm thử ở một máy khác máy phát triển. Chương 5 đối chiếu tất cả với yêu cầu đã đặt ra ở chương 1 và với các chương của môn học.

CHƯƠNG 5.

KẾT LUẬN

Chương cuối đối chiếu kết quả với hai bộ yêu cầu đã đặt ở chương 1 và với mười chương của môn học, nói thẳng những hạn chế còn lại tính đến ngày nộp, và nêu hướng phát triển đã được xếp thứ tự. Cột bằng chứng trong các bảng chỉ về mã, test hoặc lần chạy cụ thể, không về ý định.

5.1. Kết Quả Đạt Được

5.1.1. Đối Chiếu Với Yêu Cầu

Trạng thái ghi theo ba mức: **Đạt**, **Một phần**, **Chưa**. Bằng chứng "VM" là lần chạy offline từ clone sạch trên máy ảo Linux ngày 16/10/2026 (Mã nguồn 40); "macOS" là lần chạy thật qua Chrome ngày 26/09/2026 (Phần 4.2).

Mã	Trạng thái	Bằng chứng
FR-01	Đạt	`list -p` đọc offline (VM); `list -p -r` làm mới qua trình duyệt (macOS); `fetch_catalog` Mã nguồn 24
FR-02	Đạt	`fetch -p 280 --cascade --toc` (macOS); commit `cc06f45`; `test_path.py`
FR-03	Đạt	`fetch -c 892 --toc` (macOS); bốn bước của `extract_transcript` ở Phần 3.6; `test_course.py`
FR-04	Một phần	`fetch -l ID` được nhận nhưng in "not yet fully supported": lab chỉ được thu thập qua course chứa nó (`process_lab`, Mã nguồn 15); `md -l` và `list -l` chạy được
FR-05	Đạt	`extract_course_metadata` so `datePublished` (Mã nguồn 30); `test_metadata_skips_when_date_published_unchanged`
FR-06	Đạt	`md -c 892 --toc` sinh note ở Mã nguồn 34 (VM); `generate_markdown` ghi đè ở Path, Course, Lab
FR-07	Đạt	`search vertex` tìm thấy path 280 và course 892 (VM); `Index.search` với `--field`
FR-08	Đạt	`reindex` dựng lại `index.json` (VM); `Index.rebuild` Mã nguồn 32; `test_store.py`
FR-09	Đạt	`process_document` tải vào `materials/courses/<id>/` với cookie của trình duyệt; commit `b4bd646` (macOS)
FR-10	Một phần	`Course.generate_prompt` sinh `<id>-prompt.json`, gọi được từ TUI (`g`), chưa có lệnh CLI; tạm ngừng theo ghi chú ở Bảng 3
FR-11	Đạt	`skills-scraper-tui` mở menu màu và thoát sạch (VM); Hình ảnh 6

Mã	Trạng thái	Bằng chứng
NFR-01	Đạt	ba cơ chế ở Phần 4.2, có test
NFR-02	Đạt	JSON UTF-8 có thụt lề, Markdown thuần; `git diff` trên vault là cách phát hiện thay đổi
NFR-03	Đạt	ba lớp cấu hình ở Mã nguồn 41, `config.example.yaml` có chú thích
NFR-04	Đạt	`try/except` theo từng activity; loại lạ giữ nguyên và báo (Mã nguồn 21); bỏ qua course có in lý do
NFR-05	Đạt	`write_text` nguyên tử (Mã nguồn 25); `test_write_json_leaves_no_temp_files`
NFR-06	Đạt	`uv sync` rồi `pytest` xanh từ clone sạch trên hệ điều hành khác máy phát triển (Mã nguồn 40); `grep -rn "/Users/" src` rỗng

Bảng 10: Trạng thái 11 yêu cầu chức năng và 6 yêu cầu phi chức năng

Hai mục "Một phần" đều có cùng một nguyên nhân: ở v2.0.0 chúng cũng chưa hoàn chỉnh, và trong phạm vi đồ án sinh viên ưu tiên làm cho những gì đã có chạy đúng với site hiện tại hơn là mở thêm tính năng. Cả hai được giữ nguyên trong mã với thông báo tường minh thay vì bị giấu đi.

5.1.2. Đối Chiếu Với Chương Bài Giảng

Chương	Nội dung	Bằng chứng trong mã
01 *[8]*	Môi trường Python	`pyproject.toml`, `.python-version`, `uv.lock`; Mã nguồn 3
02 *[9]*	Kiểu dữ liệu	`dict` lồng modules/steps/activities; f-string; `tuple` làm khoá sắp xếp; Mã nguồn 18, Mã nguồn 19
03 *[10]*	Rẽ nhánh	dispatch bằng `dict` theo loại activity; bỏ qua theo `datePublished`; Mã nguồn 20
04 *[11]*	Vòng lặp	phân trang `while` có cầu chì `MAX_PAGES`; ba tầng `for`; Mã nguồn 24
05 *[12]*	Hàm	tham số mặc định, `**kwargs`, `lambda`, `@staticmethod`, đệ quy `_parse_lesson_item`; Mã nguồn 22
06 *[13]*	Module, package	`src/skills_scraper/` với ba sub-package, `__init__.py`, hai entry point; Mã nguồn 6
07 *[14]*	Ngoại lệ	bắt theo activity, `try/except/raise` khi ghi nguyên tử, `KeyboardInterrupt`; Mã nguồn 25
08 *[15]*	Hướng đối tượng	`Serialize`, `BaseEntity`, `Collection`, `@property`, `super()`, ghi đè, composition; Hình ảnh 3
09 *[16]*	Giao diện đồ hoạ	không dùng, có lý do ở Phần 2.6; TUI thay thế
10 *[17]*	Cơ sở dữ liệu	không dùng, có lý do ở Phần 2.6; file JSON và `index.json` dựng lại được

Bảng 11: Mười chương của IE221 và nơi chúng xuất hiện trong mã

5.2. Hạn Chế

Tính đến ngày nộp, công cụ còn những giới hạn sau, tất cả đều được nói ra trong mã hoặc tài liệu kèm repo:

- Cần một phiên đăng nhập thật nên không chạy hoàn toàn không người trông; `--headless` chỉ dùng được sau khi profile đã đăng nhập.
- Chưa thu thập lab độc lập (FR-04) và chưa có lệnh CLI cho tệp prompt (FR-10).
- Chưa thử lại khi mất mạng (P10 ở Phụ Lục A.3): một lần rớt kết nối giữa `--cascade` dừng course đang chạy, chạy lại lệnh sẽ tiếp từ course đó nhờ idempotent.
- `print()` còn 60 chỗ trong `model/` thay cho `logging`; `ruff check` (theo PEP 8 *[18]*) còn 90 cảnh báo, đa số về kiểu (`implicit-optional`, `blind-except`, thứ tự import), chưa có lỗi logic.
- Chất lượng transcript phụ thuộc phụ đề của site; tên file Markdown lấy từ tiêu đề nên khi Google đổi tiêu đề thì liên kết trong vault gãy; checkbox tiến độ người học tự đánh bị ghi đè khi sinh lại Markdown.
- Partner portal không còn kiểm thử được vì sinh viên không còn quyền truy cập.
- Nội dung transcript và tài liệu thuộc về Google: vault công khai chỉ nên chứa cấu trúc (`--toc --no-transcript`).

5.3. Hướng Phát Triển

Theo thứ tự nên làm, mỗi mục là một hoặc vài commit nhỏ trên cùng kiến trúc, không mục nào đòi sửa `model/` về căn bản:

1. Thử lại khi mất mạng (P10) và tái dùng cửa sổ Chrome đang mở qua `debuggerAddress` (P11), để `--cascade` dài hơi chạy ổn.
2. Bundle OKF đầy đủ: lệnh `index` sinh `index.md` cho mỗi thư mục và `log.md` ở gốc vault; khi đó `docs/data.md` và `docs/output.md` viết lại cho khớp mã.
3. Tên file dạng `<id>-<slug>.md` để liên kết không gãy khi đổi tiêu đề; `print()` sang `logging`; dọn các cảnh báo `ruff`.
4. Thu thập lab độc lập và lệnh `prompt` trên CLI, khép lại hai mục "Một phần".
5. Portal khác (AWS Skill Builder, NVIDIA DLI) bằng cấu hình và lớp con ghi đè phần phân tích trang, đúng cách `Paths` ghi đè `API_URL`.
6. Dùng vault làm kho tri thức cho AI agent qua plugin Obsidian hoặc MCP, với tệp prompt JSON làm đầu vào tóm tắt; đây là nơi khung CODE đi hết bước **Distill**.

5.4. Lời Kết

Đồ án xuất phát từ một nhu cầu thật của sinh viên trong công việc và được giải bằng một công cụ nhỏ, cố ý nhỏ: thu thập nội dung Google Skills về file JSON và sinh một vault Markdown cho Obsidian, với hai runner trên cùng một lõi. Về mặt học thuật,

mười chương của môn *IE221 – Kỹ Thuật Lập Trình Python* đều có chỗ đứng trong mã (Bảng 11), kể cả hai chương được quyết định không dùng với lý do rõ ràng; về mặt kỹ thuật, 15 trong 17 yêu cầu đạt, hai yêu cầu còn lại đạt một phần và được nói thẳng (Bảng 10).

Điều sinh viên mang theo từ đồ án không phải là bản thân scraper, vốn sẽ phải đổi theo site, mà là ba thói quen: dữ liệu để lại phải sống lâu hơn công cụ, lỗi ồn ào tốt hơn lỗi im lặng, và mỗi quyết định dù là "không làm" cũng phải có lý do ghi lại được. Đó cũng là cách sinh viên đang xây các thành phần nhỏ, không trạng thái cho những hệ thống lớn hơn trong công việc hiện tại.

PHỤ LỤC A

Nguồn Gốc Và Tư Liệu

A.1 Nguồn Gốc Mã Nguồn

Ứng dụng là branch `refactor-ie221` của repo `github.com/ggcta/skills-scrapper`, tách từ tag `v2.0.0` (commit `ac49ba2`, 18/02/2026), công trình trước đó của cùng tác giả, bắt đầu từ 05/01/2025 dưới tên Cloud Skills Boost helper. Dòng chính của repo đã chuyển sang lõi Go và giao diện Tauri (tới v2.17.0); báo cáo này chỉ xét phần Python và chỉ dùng dòng chính làm nguồn tham khảo cho các thay đổi của site.

Tính đến bản nộp, branch có 22 commit sau `v2.0.0`, HEAD là `45c7d0a` (27/09/2026). Phiên bản gói trong `pyproject.toml` là `2.0.1`; `CHANGELOG.md` trong repo mô tả các thay đổi dưới mốc `py/v2.1.0 (unreleased)`, tiền tố `py/` vì dòng chính đã dùng `v2.1.0` trở lên cho phần Go; tag tạm `py/v2.1.0-rc1` đặt ở commit `5e161e7`. Mã nguồn nộp kèm báo cáo là bản zip của branch này, không có `.venv`, `data/` và vault.

A.2 Danh Sách Commit Refactor

Kết quả `git log --oneline v2.0.0..refactor-ie221`, mới nhất ở trên; 16 commit đầu (24/09/2026) theo đúng kế hoạch chuẩn hoá, 6 commit sau là sửa nhỏ khi chạy thật trên macOS.

```
45c7d0a BASE_URL
18c4a22 BASE_URL
002091f Update .gitignore
4641120 Update tui.py
28ab46e Update cli.py
8b75822 id size: 6 digit
5e161e7 docs: README for the Python line, CHANGELOG, justfile
1cddcfd tests: parsers, renderer, front matter and store, all offline
cc06f45 fetch -p ID --cascade: the path, then every course in it
b4bd646 materials under the vault, every file written atomically
14ee352 no database: JSON files are the truth, index.json is a ledger
23d5a63 frontmatter: valid YAML, OKF v0.2 superset
4e2e383 browser: one flag list, headless=new, profile reused by
default
1167943 path: ql-contents-menu decides what is a course and what is a
lab
9cfcd03 activities: dispatch by kind, html_bundle joins, unknown kinds
survive
ae21659 catalog lists through the browser, one fetch_catalog for all
three
873a315 cli: import Lab, md --lab has been a NameError since v2.0.0
9cc37c4 every page goes through the signed-in browser
bc4179f config.yaml is the new settings.py
f864415 drop the Flask web UI and the dead code around it
84c5d98 move app/ into src/skills_scraper, real package now
879de08 uv: pyproject + lock, bye requirements.txt
```

Mã nguồn 42: `git log --oneline v2.0.0..refactor-ie221`, 22 commit

A.3 Bảng Port Từ Dòng Chính

Mười ba việc được xác định khi đọc dòng chính (v2.1.0 tới v2.17.0) để đưa về Python; cột trạng thái đối chiếu với branch tại HEAD `45c7d0a`.

Mã	Nội dung	Trạng thái	Commit
P1	cấu hình bằng `config.yaml` và biến môi trường, bỏ đường dẫn cá nhân	đã làm	`bc4179f`
P2	mọi trang qua trình duyệt, đăng nhập một chỗ	đã làm	`9cc37c4`
P3	dispatch activity, thêm `html_bundle`, giữ loại lạ	đã làm	`9cfcd03`
P4	path đọc từ `ql-contents-menu`	đã làm	`1167943`
P5	cờ Chrome mới, tái dùng profile	đã làm	`4e2e383`
P6	frontmatter hợp lệ, khoá OKF	đã làm	`23d5a63`
P7	ghi tệp nguyên tử	đã làm	`b4bd646`
P8	JSON là nguồn sự thật, bỏ TinyDB	đã làm	`14ee352`
P9	tài liệu vào `materials/courses/<id>/`	đã làm	`b4bd646`
P10	thử lại khi mất mạng	chưa	—
P11	tái dùng Chrome qua `debuggerAddress`	chưa	—

Mã	Nội dung	Trạng thái	Commit
P12	fetch path lan xuống course và lab	đã làm	`cc06f45`
P13	xoá mã chết (Flask, `data_management.py`, `tasks/`)	đã làm	`f864415`

Bảng 12: Mười ba việc port từ dòng chính và commit thực hiện

A.4 Kịch Bản Demo

Năm bước, 5 tới 7 phút, theo thứ tự offline trước, mạng sau, để nếu mạng hay đăng nhập có vấn đề thì phần cốt lõi vẫn đã được thấy.

Bước	Lệnh	Điều cần chỉ ra
1	`uv run skills-scraper list -p`	đọc offline từ `data/`, không mở trình duyệt
2	`uv run skills-scraper md -c 892 --toc`	note sinh ra trong vault, frontmatter OKF, mở trong Obsidian thấy backlink tới path
3	`uv run skills-scraper fetch -c 892 --toc`	phiên đăng nhập một lần, log từng activity; chạy lại lần hai thì bỏ qua và in lý do
4	`uv run skills-scraper-tui`	cùng lõi, giao diện khác; thoát bằng `q`, Chrome đóng theo
5	vòng `for` shell, `jq`, `git diff --stat` (Mã nguồn 37)	ghép được với công cụ khác mà không cần API

Bảng 13: Kịch bản demo năm bước

A.5 Chạy Từ Bản Nộp

Giải nén, rồi trong thư mục gốc của repo:

```
uv sync                                  # Python 3.12, thư viện, hai
lệnh skills-scraper*
uv run pytest                            # 15 test offline, không cần
mạng
uv run skills-scraper --help
CSB_DATA=docs/samples/data CSB_VAULT=/tmp/vault \
  uv run skills-scraper md -c 892 --toc   # sinh một note từ dữ liệu
mẫu trong repo
cp config.example.yaml config.yaml      # tuỳ chọn: nơi đặt data/ và
vault
uv run skills-scraper list -p -r        # mở Chrome, đăng nhập một lần
```

Mã nguồn 43: Các lệnh để chạy từ bản zip nộp kèm

A.6 Công Cụ Sử Dụng

Python 3.12, uv, Selenium 4, Beautiful Soup 4, requests, PyYAML, pytest, ruff, Chrome, Obsidian; báo cáo và slides soạn bằng Typst 0.15.1 với theme `uit-theme` của cùng tác giả, dựng bằng `just`; Claude (hỗ trợ rà soát mã và soạn tài liệu, mọi đoạn mã và số liệu trong báo cáo đều được kiểm lại trên repo).

Tài Liệu Tham Khảo

[1] E. S. Raymond, *The Art of UNIX Programming*. Addison-Wesley, 2003.

[2] T. Forte, *Building a Second Brain*. Atria Books, 2022.

[3] Google Cloud Platform, "Open Knowledge Format Specification v0.2". [Online]. Có tại: https://github.com/GoogleCloudPlatform/open-knowledge-format

[4] Astral, "uv Documentation". [Online]. Có tại: https://docs.astral.sh/uv/

[5] Python Packaging Authority, "Python Packaging User Guide". [Online]. Có tại: https://packaging.python.org/

[6] Selenium Project, "Selenium Documentation". [Online]. Có tại: https://www.selenium.dev/documentation/

[7] L. Richardson, "Beautiful Soup Documentation". [Online]. Có tại: https://www.crummy.com/software/BeautifulSoup/bs4/doc/

[8] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 01: Giới thiệu Python và môi trường". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[9] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 02: Kiểu dữ liệu". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[10] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 03: Cấu trúc rẽ nhánh". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[11] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 04: Vòng lặp". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[12] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 05: Hàm". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[13] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 06: Module và package". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[14] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 07: Xử lý ngoại lệ". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[15] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 08: Lập trình hướng đối tượng". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[16] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 09: Giao diện đồ hoạ (tkinter, PyQt)". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[17] Nghi Hoàng Khoa, "IE221 Kỹ Thuật Lập Trình Python, Chương 10: Cơ sở dữ liệu (MySQL, MS SQL)". Trường Đại Học Công Nghệ Thông Tin, ĐHQG-HCM, 2026.

[18] G. van Rossum, B. Warsaw, và A. Coghlan, "PEP 8: Style Guide for Python Code". [Online]. Có tại: https://peps.python.org/pep-0008/